

(4th, 5th, 9th)

⇒ How can we use it?

Page No:
Date: 13/03/25

a) Total no. of possible partitions

$$B_n = \sum_{k=1}^n S(n, k)$$

$$S(n, k) = \binom{n}{k} B_k$$

↓

- 1) Non-overlapping
 2) Non-empty (at least 1)
- } partitions

Assume $(n-1)$ data points, k partitions I have

$$\Rightarrow (k S((n-1), k) + S((n-1), (k-1))) = S(n, k)$$

External Evaluation Method

b) All pairs sent to,
 $\Rightarrow TP, TN, FP, FN$

$$\text{Rand Index} = \frac{TP + TN}{TP + TN + FP + FN} \quad ({}^nC_2)$$

To Do List

Not To Do List

↑ Success in not to do list doesn't matter
 as it is very big

here TN is very large

$\Rightarrow TP$ to be removed? $(\dots)(X)$ ← here he considers TP

$$\text{Jaccard Index} = \frac{|\text{Intersection}|}{|\text{Union}|}$$

$$= \frac{TP}{TP + FP + FN}$$

Here Intersection & Union is taken but ideal & the new cluster formed

predicts on small positive cases (if $FN \uparrow$)

Precision = $\frac{TP}{TP + FP}$
 for together pairs
 ↑ in the cluster
 out of which how much correct

out of all positive predicted how many correct
 Recall = $\frac{TP}{TP + FN}$

it talks of how similar

(i.e. Precision or recall)

We can't go either of them separately
 Perfect Score if all pairs marked as positive in my cluster

out of all positive cases how many pairs that exists in ideal

F1 means (Harmonic Mean)

$$\frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

if one goes
down it goes
down significantly

0-1

↑ note: it is by (%)
assumed to be 0

we have Fowlkes & Mallows Index (G_M)

$$\sqrt{\text{Precision} \times \text{Recall}}$$

Which one is better? similar measure
⇒ depends on application

Which one is best clustering method?
⇒ depends on context

Internal Measures

• (no ground truth)

• we can't go for TP, TPV, AP, FN

• go across all clusters, within each cluster, compute score

Method 1 for each data point (S)

evaluation
measure for
that clusters

$$S = \sum_{i=1}^K \frac{1}{|C_i|} \sum_{j \in C_i} S(x_j^i)$$

[-1, 1]

S branches into:

- Average intra cluster distance: $A(x_j^i)$
- Average distance to nearest cluster: $B(x_j^i)$

$$\sum_{\substack{m \in C_i \\ m \neq j}} d(x_j^i, x_m^i)$$

$$\min_{\substack{n=1,2,\dots,K \\ n \neq i}} \left(\frac{1}{|C_n|} \sum_{y \in C_n} d(x_j^i, y) \right)$$

Silhouette Coefficient

→ (-1, 1)

Page No:

Date: / /

$$S = \frac{B(x_j') - A(x_j')}{\max(B(x_j'), A(x_j'))}$$

$B(x_j') \uparrow$ for points very well placed inside cluster (1)

$A(x_j') \downarrow$ for " not placed very well (-1) → $B(x_j') \downarrow$

$A(x_j') \uparrow$

What's the problem with this? ~~right~~ and ~~no~~ ~~time~~

→ all pair wise dist req. after clusters formed (computation high)

Method 2: Go for each cluster

(This one tells us how tight & well separated the clusters are)

$C_1, C_2, \dots, C_k$

C_1

C_2

$\vdots$

C_k

Diameter

• how to define distance bet. 2 clusters?

(1) → any choice

→ for points we had specific

(2) we calculate the diameter as well

It tells us spread out of the cluster {maximum distance within cluster}

• Pick up lowest inter cluster distance

distance

Highest diameter

called ~~distance~~ Index

$$= \frac{\min \text{ inter-cluster}}{\max \text{ intra-cluster}}$$

look at centroid something

C_i, C_j

Avg Intra cluster distance: $A(C_j)$

(henceforth, go for any)

$$\frac{A(C_i) + A(C_j)}{2}$$

$$d(C_i, C_j)$$

for computing distance bet. 2 clusters & then do (A) again

Q. Should we go with 1 clustering method? (No)

→ can I club together multiple clustering algo? (Yes)

Each A_i gives diff clusters (or may be same)

Page No:

Date: / /

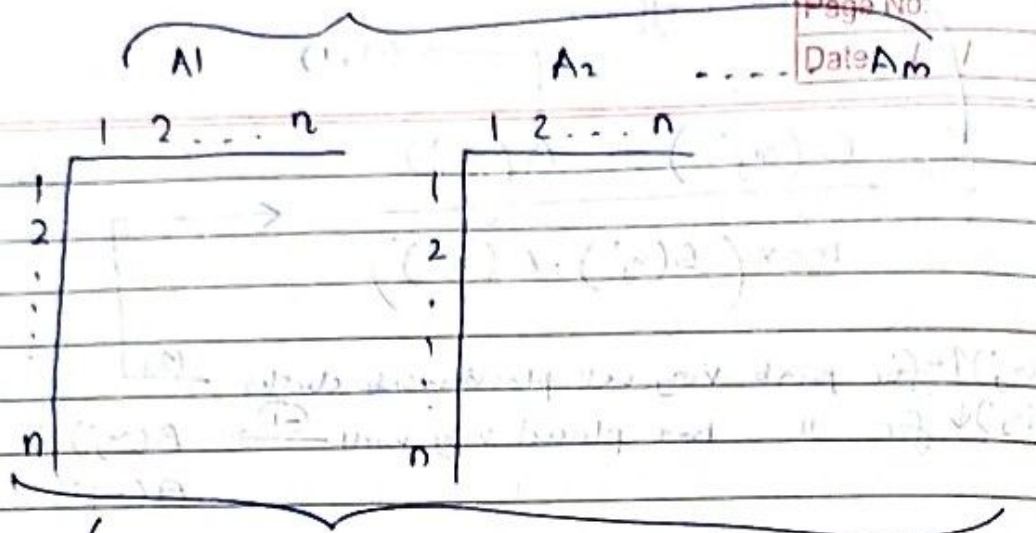
Combining

Multiple

Clustering

Algorithms

→ Clusters
Ensemble



• What can we infer from this for each pair?

Method 1 $\Rightarrow$ Average for the final matrix
: Similarity measure

$$\text{Distance} = \frac{1}{\text{similarity}}$$

• Go through all m matrices $\rightarrow$ by taking avg

• Compute the final similarity matrix

• then use any clustering algo

Method 2 $\Rightarrow$ Graph clustering algo

$\downarrow$ here we can go

i) Build graph for graph

ii) With distances as the weight

iii) Since now we have got the whole graph,

we can apply any clustering algo (e.g. remove edges etc) to

graph to get desired no. of clusters

for the graph as like in

what graph looks $\rightarrow$

graph looks like

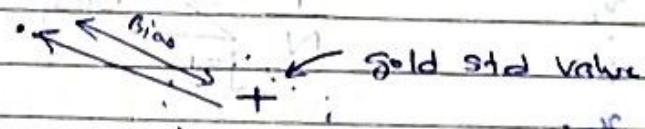
(A)

Page
data

- a) Quiz 2 : April 2nd/3rd , Project : April 2nd Week
 Mid Sem : Small Weight (Friday)
 End Sem : Open Notes

Bias Variance Trade

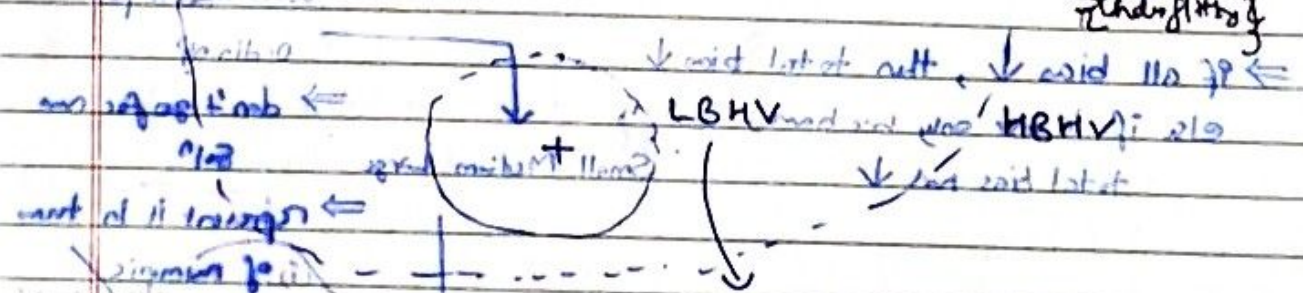
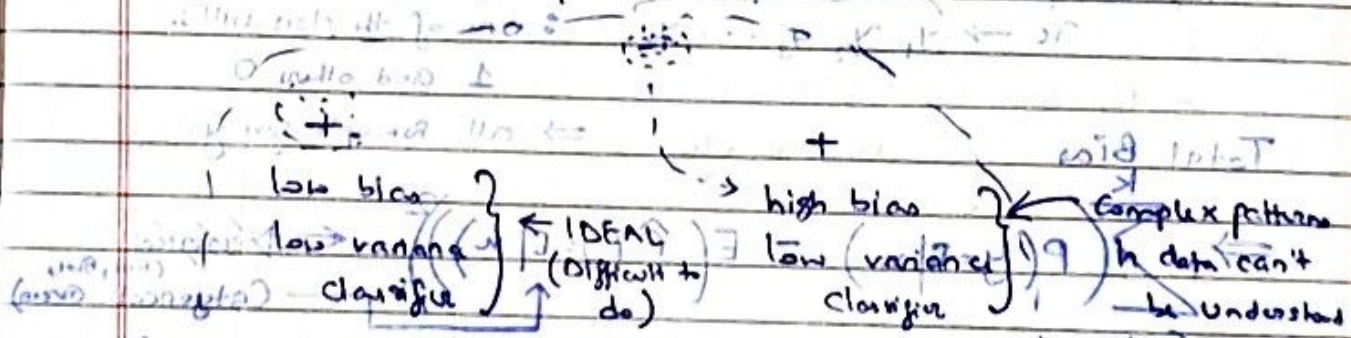
- b) • Making predictions in 2D-space
 • Bias : how far away from gold truth



→ how unstable our predictions is

Variance : measures how far all points (to each other?)

What is multiple prediction?



• Overfitting to data → hence we can't understand the complex patterns

Case 1: y : (Build a model for x to predict y)

Single data point $\rightarrow$ $g_1 : M_1$
 $g_2 : M_2$ (Store single point account hence not reg)
 $\vdots$
 $g_m : M_m$

$\rightarrow$ Bias $= \sqrt{E((y - E(g))^2)}$

expectation $\rightarrow$ expectation over all models

• Have to build the models (multiple)?
 • Random initialisation
 • Sample data set (sampling with replacement)

$$E(\hat{y}) = \frac{1}{M} \sum_{j=1}^M \hat{y}_j \leftarrow \text{Non-ordinal attribute (0-100)}$$

Case 2: Multiple data points

$$\begin{matrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{matrix} \quad \frac{1}{N} \sum_{j=1}^N \text{Bias}_j^2 \leftarrow \text{What does it represent?}$$

Bias² for database (Test)

Classification hard classes indicate what?
 Single Data Point $x \rightarrow y_1, y_2, y_3, \dots, y_k$
 $\rightarrow$ for each class considered
 $\rightarrow$ each indicates probability
 $\rightarrow$ x belongs to each k classes partially like fractions
 $\rightarrow$ one of the class will be 1 and others 0

Total Bias $\Rightarrow$ all examples give y
 $\Rightarrow$ What does this represent?
 $\sum_{C=1}^K (P(y=C|x) - E(\hat{P}(y=C|x)))^2$
 $\leftarrow$ Discrete (Red, Blue, Green)
 $\leftarrow$ Categorical (Green)

$\Rightarrow$ if all bias $\downarrow$, then total bias $\downarrow$
 else if bias only one have, total bias not $\downarrow$

Small Medium Large

What about for ordinal
 $\Rightarrow$ don't go for one $\Rightarrow$ represent it in terms of numeric

Bias types of Data Bias

Statistical bias
 dataset given is truly to be learnt

Paras bias in Machine Learning

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$\Rightarrow$ represent it in terms of numeric
 (i) of numeric
 (ii) classification
 $\Rightarrow$ otherwise, represent
 $\Rightarrow$ diff but large -
 $\Rightarrow$ mid & large - small
 $\Rightarrow$ (hence ignore the ordinal)
 Assume equal spacing with 100 labels
 Categories, which might not be true

(iii) ordinal metric: eg

Cohen's Kappa
 Quadratic

g. (splitting) where all H and T are H

penalises larger errors more heavily

V

$\Rightarrow$ E

R

(Single data point across all models)

E

What is the difference between

Taking expectation on (A)
 diff
 or

(2ab)

With weighting

→ A low variance suggests that the models generally agree on the choice

Variance : (no picture of ideal point)

$\hat{y}$: prediction made (out of predictions made)

$$\Rightarrow E[(\hat{y} - E(\hat{y}))^2] = \frac{1}{n-1} \sum_{i=1}^n (y_i - E(\hat{y}))^2$$

across all models
population variance
for sampled variance
diff data points only?

across all data points : as in all possible $\hat{y}_i$

(Test)

For a model, we look into bias + variance + error

$$y = f(x) + \epsilon$$

mean = 0, variance = σ^2
error added (built into it)

(single data point across all models)

true value has some error associated with it

Expectation of squared error

$$\# E[(\hat{y} - y)^2] \Rightarrow E[(\hat{y} - f(x) - \epsilon)^2]$$

$$\Rightarrow E[(\hat{y} - f(x))^2] + E[\epsilon^2] - 2E[(\hat{y} - f(x))\epsilon]$$

$$= (\hat{y} - E(\hat{y}) + E(\hat{y}) - f(x))^2$$

(A)
(variance of prediction)
(variance of noise)
(variance of noise)

Taking expectation on (A)

$$= E[(\hat{y} - E(\hat{y}))^2] + E[(E(\hat{y}) - f(x))^2] + 2ab$$

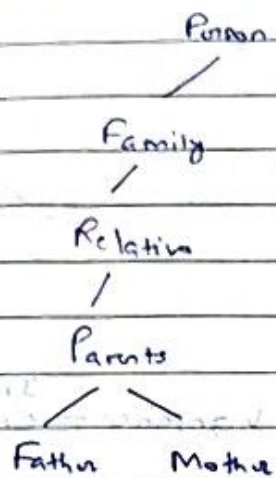
(variance)
(bias)
(bias)
(bias)

$$(2ab) : E[2(\hat{y} - E(\hat{y})) (E(\hat{y}) - f(x))]$$

(This is mean value)
As expectation over
All models remain constant & same with $f(x)$ i.e. constant

Types of classification pbs

- a) Hierarchical classifications — labels are not list
(not discussed) ex. — more tricky to evaluate



(d) Fuzzy classification

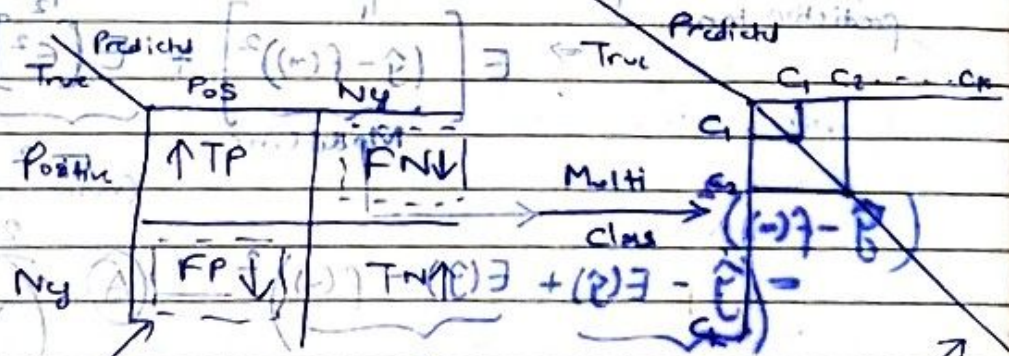
— membership across all

labels

(Sort of probability)
distribution

- b) Binary — 2 out of the class
Multiclass — Discussed
Multilabel — More than 1 class

- c) After a model is trained, given a dataset, it will predict outputs based on either of 4 classes:



error

Binary

Confusion Matrix

Diagonal entry true prediction

whole matrix should

- d) Depends on the classification problem, (another) concentration ground

either low FN or medium (FN & FP)

is okay

(See) (See)

e) example Medical Test

FP : person doesn't have disease, but predicted to have

FN : person has disease, but predicted not to have

Which one should be low: FP or FN?

→ depends on context like type of disease, medical treatment

example Spam Email

FP : not spam email but went to spam

FN : actually spam " but predicted not spam

Which one should be low? FP or FN?

example. What classification problem trying to solve?

⇒ Text Batter (play long innings) } ← is the ball risky?

T20 Batter (score runs quickly)

↑ can I hit the ball & get runs?

(Is it swashy?)

TP

TN

FP

FN

risky ball; but predicted not risky

Scoring but pred: FN
- Did not scoring

How to measure classification? Similar

f) if ^{model} accuracy (~) it doesn't imply performance is same as FN is high

(measures how often the model's predictions are correct overall)

How can we get it high?

(if FN is 90% make all FN) (Sntg)

$$\text{Accuracy} = \frac{TP + TN}{\text{All predictions}}$$

To get accuracy

high → we can make predictions when we are only confident to get TP ↑

⇒ One single measure doesn't give overall idea of the model

example Model confidence ignored

(high pb ⇒ the class)

C₁ C₂ C₃
P₁ P₂ P₃

Deterministic Decoding

M1 →

0.90

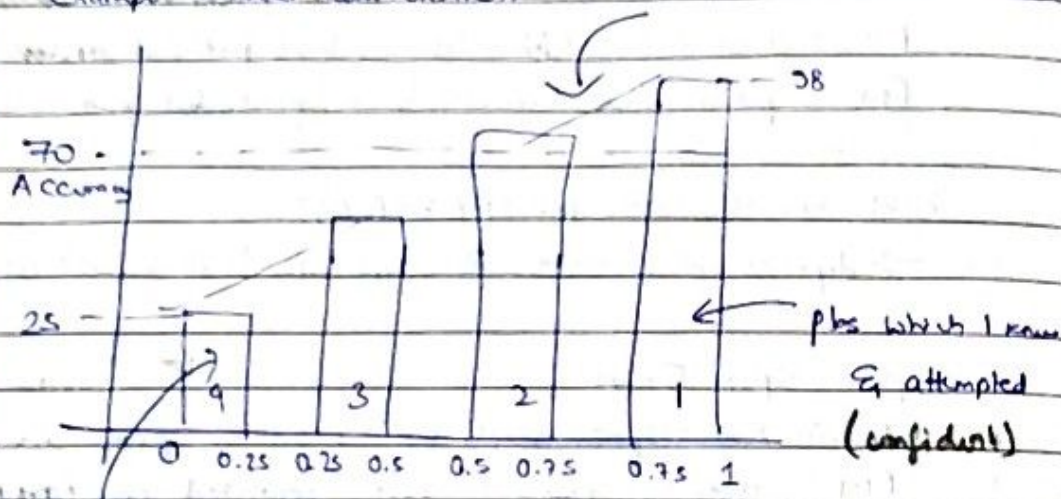
M2 →

0.75

Are these 2 same? even if they report C₂ (No)

example Model Calibration

Well Calibrated model



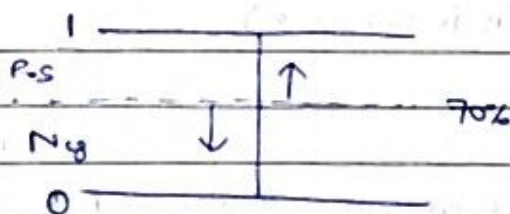
problems which

Confidence

I didn't know
I attempted
(not confident)

Higher confidence $\Rightarrow$ Higher
Chance of Success

example Threshold Tuning



here no idea of how
accuracy will change if
we go above the threshold
or if you below

What is threshold

here for?

To classify objects

Problems relation to Precision & Recall

(most are
negative)

$\rightarrow$ High Precision

$\rightarrow$ High Recall

what does it not cover?

like we can make very rare positive
cases but still have high precision

everything is

positive

(CHECK!)

Specificity = $\frac{TN}{TN+FP}$ ← Since if mark all negative & FN not considered, it gives high value

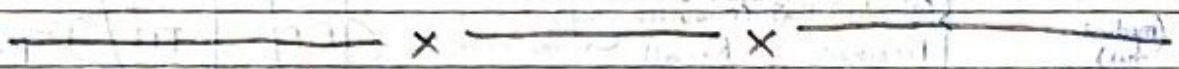
- don't miss any negative class
- here ignoring false negative mark
- (So ~~if~~ all negative ~~cases~~)

Negative Predictive Value : $\frac{TN}{TN+FN}$ ← predict only when Confident (negative part)

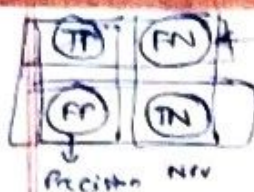
Make
predictive
only when
Confident

- very small no. of FN. then this value is high
- Since predict negative values mostly

⇒ if we mark then positive as well it won't matter



#



Recall

Specificity

Evaluation Measure

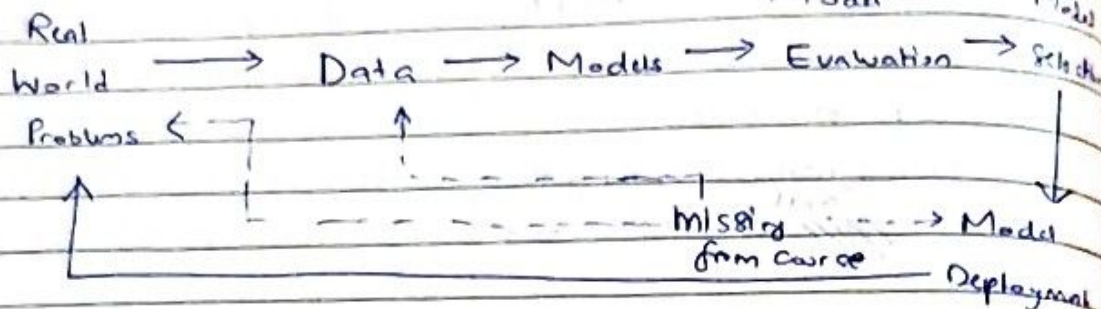
FPR = 1 - Specificity

FNR = 1 - Recall

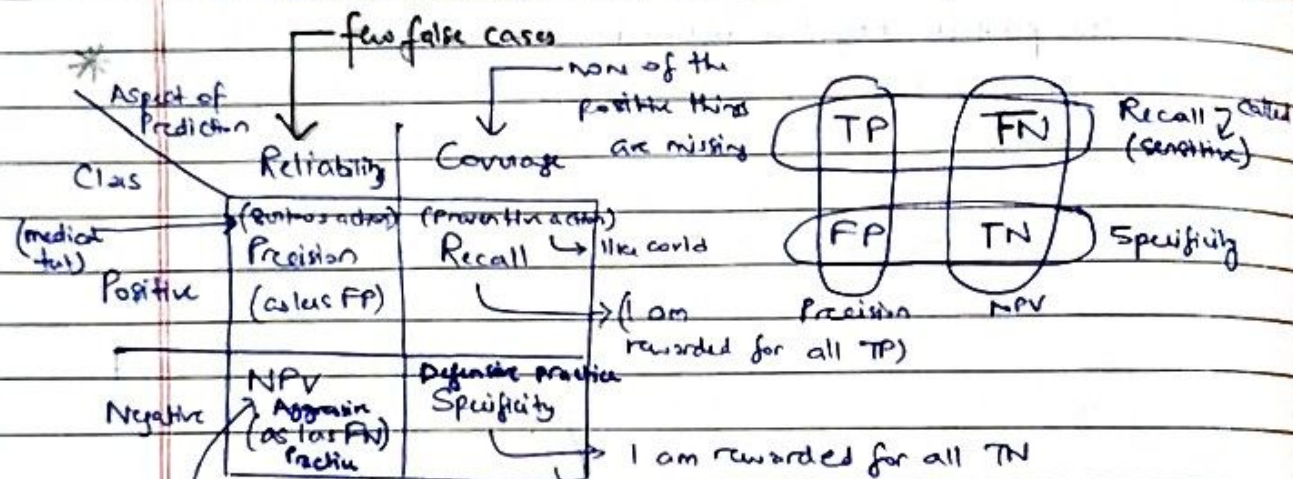
Page No:

Date: 20/02/25

Big Picture



- How to prepare data (skipped) — data cleaning, data prep
- No domain knowledge — world's real world pb



- aggressive practice
- very much reliability for negative class
- I might miss many neg
- when I say neg, it is neg
- example: Mass Recruitment (??)
- I should not miss any neg
- ex. judicial system (no innocent should be punished)

Too better

→ Type 1 error : affects precision

$$(i) FPR (False positive Rate) = 1 - \text{Specificity} = \frac{FP}{FP + TN}$$

$$(ii) FNR (False negative Rate) = 1 - \text{Sensitivity}$$

$$\rightarrow \text{Type 2 error : affects recall} = \frac{FN}{TP + FN}$$

- * The table talked about what measures I will talk if I want to check for a classifier model with reliability or coverage for negative or positive predictions (correctness)
- # Reliability: how much can you trust the model's prediction?
- # Coverage: how well does the model capture all relevant cases?

helps evaluating a model performance, especially when classes are imbalanced (i.e. when positive cases are rare compared to negative) req high Recall, high precision

good for focusing on positive class performance

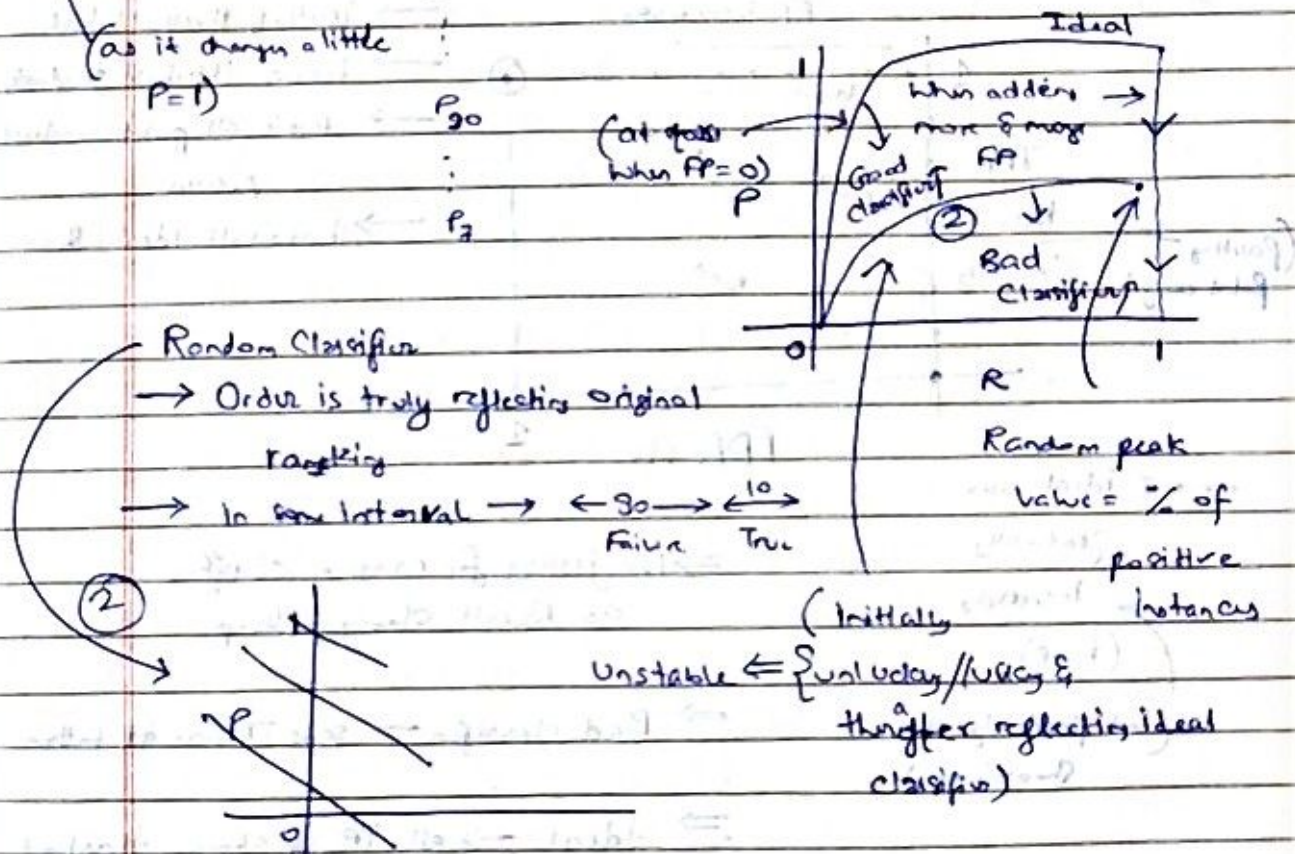
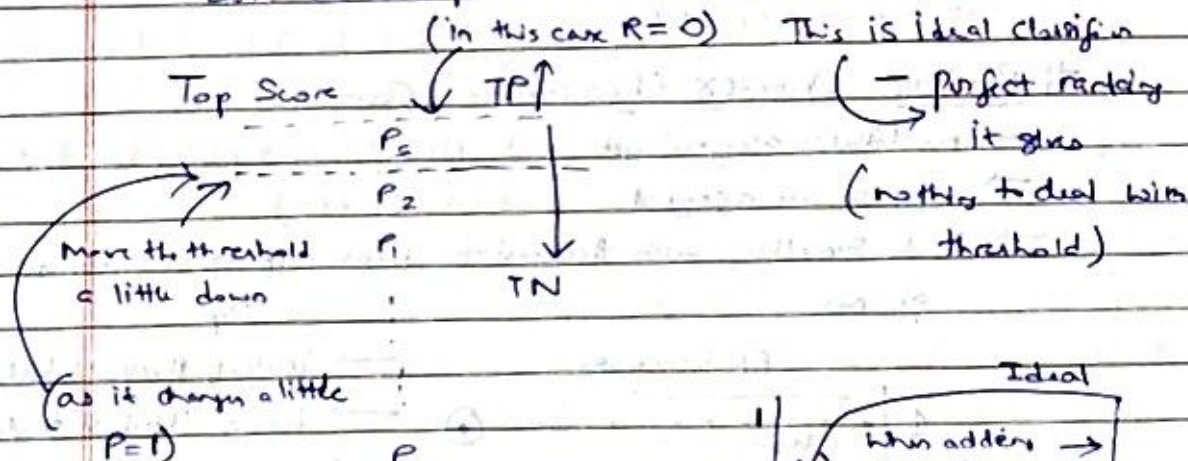
Precision Recall Curve - closer to top right the better model

- Score for each point - $[0, 1]$

Forecast: How to decide threshold i.e. $0.8 \rightarrow 1$? or $0.6 \rightarrow 1$?

- for each threshold, we have precision & recall $\leftarrow$ this tells for only that threshold (not as in whole classification performance)
- (So for that vary the threshold to get general idea)

- Sort the data patch based on Score



Here we talk about area under the curve

Ideal = 1 $\leftarrow$ (AUC) $\rightarrow$

(a good PR curve is much greater AUC)

(NOTE: classifier that has a higher AUC on the ROC curve will always have a higher AUC on the PR curve as well)

Page No:
 Date: / /

for good classifier - initial graph same as ideal cluster

for bad classifier - It is unstable initially extremely

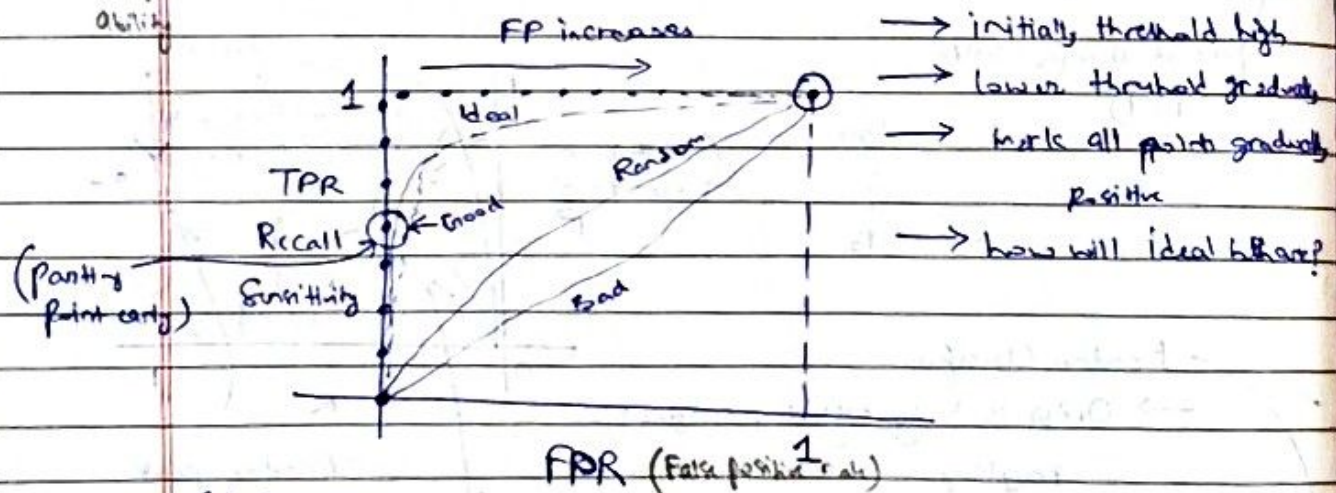
- Enam Sweet Spot
- Evaluation at low recall: ~~more~~ all four same
 - When ^{low} recall: Ideal & good same
 - high recall: All four diff
 - Complete high recall: only Ideal has ① precision & random good ~~bad~~ together
 - extreme: all four same

Enam Sweet Spot: The point where adding more models doesn't significantly improve accuracy but increases computation

used for balanced datasets

Receiver Operator Characteristic Curve

- good for recall
- no labels assigned yet
 - only scores assigned
 - So do something with threshold to know if good classifier or not
- Steps



...: ideal curve

gradually

increases

(here)

(but previously it is shooting up)

⇒ No jittery for random classifier as Recall slowly builds up

⇒ Bad classifier - says TP are at bottom

⇒ Ideal → all TP up above threshold & TN below

⇒ Here good classifier picks up FP very quickly compared to poor curve (Precision Recall curve)

(for each possible threshold, there is a corresponding recall & precision) → What is threshold INPRC? → Is it same point? (No) → is it like for this threshold, PRQ is this?

i) Increasing the threshold might increase precision (fewer false positives) but it could lower recall (more false negative)

ii) Goal INPRC is to find a threshold that either minimises FPR or FN (depending on context)

(iii) ROC → depict tradeoff between specificity & sensitivity

(iv) Positive prediction accuracy $\equiv$ Precision
Positive $\equiv$ completeness $\equiv$ Recall

Topics Covered Are:

a) K-Means & its variations

~~1~~ Spherical K-Means — how is centroid updated? cluster assigned?

~~2~~ Constrained K-Means — it's just the distance

~~3~~ K-Means++ — how is the ~~initial~~ points that are far from current centers assigned?

~~4~~ With Δ inequality

~~5~~ Privacy Preserving K-Means

~~6~~ Global K-Means

~~7~~ Different cases of K-Means repeat?

~~8~~ Alkhu's Methods — how is distance calculated?

~~9~~ Anti-clustering - K-Means

→ (Check the proof properly from notes)

b) ~~1~~ properties of clustering

(i) scale invariance

(ii) η chains property

(iii) consistency

~~2~~ clustering algorithms

(i) K-clustering

(ii) C-distance

(iii) p^* distance

~~3~~ Clustering Measurement

• External Evaluation Method — Rand, Jaccard, F1

• Internal " "

~~4~~ Bias Variance Trade Off

$$(a, b) = ((a_1, a_2, \dots, a_n), (b_1, b_2, \dots, b_n))$$

Page No.

Date: / /

Task To Do!

(e) Types of classification pb
Evaluation method for each class

(i) Read Note

(ii) Read the Extra Materials

(iii) Revise //

Extra Points Covered

a) WSSSE:
$$\sum_{j=1}^k \sum_{x \in c_j} (x_i - \mu_j)^2$$

b) K-Means ensures that cluster gets better at each step

↓

How? - As in step 2, at every iteration we try to assign to a cluster to which

$$\min_j (x - c_j) \Rightarrow \text{New WSSSE} < \text{Old WSSSE}$$

also in step 3, when we update the cluster after assignment, the update ensures that the new centroid/representation is moved towards mean $\Rightarrow$ within cluster distance for each point from its centroid reduces.

c) Spherical K-Means: $\hat{u}_i \leftarrow$ unit vector

↑

(max)

we try to max $c_i \cdot \hat{u}_i$ i.e. for that i for which it will be maximum

distance squared

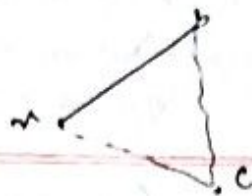
d) Seed selection in K-Means++ via:
after first seed selection

$$\text{Prob}(p_i) = \frac{\text{Score}(p_i)}{\sum \text{Score}(p_i)}$$

max prob

or taken as next centroid

ded



e) Elkan's Method

5 matrices: upper bound } ← is it computed for current assignment?
 lower bound }
 membership }
 distance to each centroid }
 Smallest distance centroid to current centroid

If $d(c_j) > 2d(u_i) \rightarrow \textcircled{X}$ ← pick the min distance to current centroid from Table

$d(c_j - c_k) > 2d(u_i) \rightarrow \textcircled{X}$ ← for each centroid c_k with current centroid

$d(u_i - c_j) > 2d(u_i) \rightarrow \textcircled{X}$ ← for each point

update

* Why random initialisation better than probabilistic initialisation?

⇒ Random initialisation

- there is a possibility that they accidentally land close to the optimal centroids
- limiting SSE to the optimal

⇒ K-Means++ initialisation

- despite being probabilistic, K-means++ prioritises points that are far away, reducing the risk of poorly placed centroids
- but in the worst case expected scenario, the SSE can go up to 8x the optimal
- This bound accounts for the fact that K-means++ has a probabilistic guarantee of good initialisation but doesn't guarantee a perfect centroids in all cases

Implications of the Probabilistic Update

The design of the probabilistic update ensures that:

- centres are well spread out, reducing the chance of poor clustering
- The algorithm is less sensitive to outliers compared to purely random initialisation
- There is a theoretical guarantee on the quality of the initialisation relative to the optimal clustering

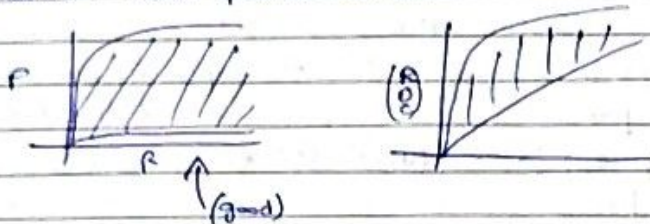
← $O(\log k)$ bound

Why harmonic mean?
 (Precision & Recall are important)
 → (it balances both precision & recall, equally weighting them both need to be high for a good score)
 Date 21/03/23

a) Another measure: F1 score (combination of reliability & coverage)
 ↓
 in terms of Recall & precision

→ random classifier - not good classifier

Cases: When positive class rate



$$F1 \text{ Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

→ F1 Score = most positive cases with high confidence

(both are → sensitive to both P & R equally important conditions)

$$F-\beta \text{ Score} = \frac{(1+\beta^2) \times \text{Precision} \times \text{Recall}}{(\beta^2 \times \text{Precision}) + \text{Recall}}$$

→ F-β Score = general form

• weights given to reliability & coverage

• if $\beta < 1$: F-β = Precision: Accuracy of the positive predictions
 if $\beta > 1$: F-β = Recall

Example (check ans)

→ Pearson Correlation (Skipped)

Invariance Prop (")

True value ↓ Predicted value ↓
 → each cell tells how many samples of C_i belong to C_j (i.e. how often a specific actual class was predicted as another) iv)

b) Confusion Matrix for Multi Class Classification

Rows: actual class

Col: predicted "

True label	Predicted label			
	C1	C2	C3	C4
C1	✓	✗	✗	✗
C2	✗	✓		
C3	✗		✓	
C4	✗			✓

What are the data points?
 ↓
 False Positive
 tells no. of instances of actual class predicted as class

TN: Sum of all elements not in main diagonal

What will happen?
 What does this mean?
 (i) if row & col same
 (ii) if col & row same
 (iii) if row + col give total no. of data points

$$\left(\sum P_i \neq N \right) \leftarrow \text{(as can be counted in multiple class for multiple)} \rightarrow$$

NOTE: for multiclass classifiers, we generally compute the measures for class (i.e. threshold) → then take average

i) Accuracy (same)

ii) Precision: (i) N

$$P_1 =$$

$$P_2 =$$

$$P_3 =$$

$$P_4 =$$

(ii) Weighted (Importance of class proportional to its size)

iii) Similarly for

iv) & same

v)

(belong to) data points actually in C_i (ideal)

predicted to be in C_i

(for multiple)

NOTE: for multiclass classification, which is the right measure?

We generally compute the measure per class ($\sum_{i=1}^K$)

→ depends on application

Page No:

Date: / /

→ then take average on Precision, Recall & F1 fundamentally theory

i) Accuracy (same defn) metrics

ii) Precision : (i) Macro (each class equally imp)

$$P_1 = \frac{TP_1}{TP_1 + FP_1}$$

$$P_2 = \frac{TP_2}{TP_2 + FP_2}$$

$$P_3 =$$

$$P_4 =$$

Each class equally important

$$\text{Macro} = \frac{1}{K} \sum_{i=1}^K P_i$$

K classes

(ii) Weighted = (importance of class proportional to its size)

$$\sum_{i=1}^K W_i P_i$$

$$W_i = \frac{P_i}{N} = \frac{\text{Row } i}{N}$$

• which is better P
→ weighted or macro
→ both have equal imp

Row corresponding to that particular class to total

iii) Similarly, for Recall → macro weighted

after a specific actual class has added as another class

iv) Same as F1-Score

{ Calculated F1 score for per class }
(check one)

Then aggregate it : (i) micro avg (ii) macro avg (iii) weighted avg

v) Micro Average — don't stick with any class

data points actually in C_1 (ideal)

predicted to be in C_1

$$\sum_{i=1}^K \frac{TP_i}{TP_i + FP_i}$$

$$\Rightarrow \frac{\sum_{i=1}^K TP_i}{\sum_{i=1}^K (TP_i + FP_i)}$$

$$\sum (\text{columns}) = \frac{\text{Correct prediction}}{N} = \text{Accuracy}$$

↳ counted in multiple class
* (for multiclass)

- each instance can have multiple correct labels
- Y_i - set of true labels for instance i
- $\hat{Y}_i$ - set of predicted labels for instance i
- N - total no. of instances

Page No:

Date: / /

c) Multi Label Classification — (Metrics calculated per instance & then averaged over the dataset)

(i) Exact Match Ratio ($\propto$ Accuracy)

No partial credit

$$\frac{1}{N} \sum_{i=1}^N I(Y_i == \hat{Y}_i)$$

ideal set of labels

• 1 if exact match

(I) gives

(ii) Hamming Loss — measures the proportion of the instance-label pairs that are misclassified

(if all 0, hamming loss high)

$$\frac{1}{N \times K} \sum_{i=1}^N \sum_{j=1}^K I(y_i^j \neq \hat{y}_i^j)$$

if belongs to K labels

doesn't work well if K is large (high recall for true negs - this)

⇒ No of labels high ⇒ High Hamming Loss

(iii) Jaccard Index — more balanced than Hamming Loss

(if Union is 1, Jaccard Index low)

$$\frac{1}{N} \sum_{i=1}^N J_i$$

$$\text{where } J_i = \frac{|Y_i \cap \hat{Y}_i|}{|Y_i \cup \hat{Y}_i|}$$

each class equally

(iv) Precision

→ Precision for datapoint, for classification, for class

(probability drawn on class low)

weight of instance

different from previous precision

$$\sum_{i=1}^N P_i$$

$$\text{where } P_i = \frac{|Y_i \cap \hat{Y}_i|}{|\hat{Y}_i|}$$

→ have to fill up? (different)

NOTE: data point based is fourth way to define precision just like macro, weighted in multi-class

• If Confusion Matrix given for multi label, is it possible to know how many data points in C_i class?

⇒ Row i doesn't tell

(V) Recall — similarly done on point label

d) Pearson Correlation between X & Y } ← for binary
 Matthews " Coefficient } → multiclass also
 ↳ $\approx$ (Pearson but on computation of predicted & actual) (smtg) exists but not discussed

ex. $A_i \leftarrow 0$ -ve class

1 +ve "

$P_i \leftarrow 1$ Predicted correctly

and for # Some derivation

(a) $A_i^2 \rightarrow A_i$ (28 summing $A_i^2 = A_i$)

(b) $A_{ij} \rightarrow A_{mean}$

↳ all ~~xxx~~ actual positive

$P_{ij} \leftarrow$ All predicted positive

→ i.e. true instances for class i

Note: $S_i \leftarrow$ Support for class i

Multiclass Classification Metrics

1) Multiclass Confusion Matrix

→ Averaging Strategy for Binary Metrics

(i) Macro average = $\frac{1}{C} \sum_{i=1}^C M_i$ Where M is the metric

each class
equally important

(ii) Micro average : Aggregate the contributions of all classes then compute

weight each

instances equally

avg globally

Micro Precision = $\sum_{i=1}^C TP_i$

$\sum_{i=1}^C (TP_i + FP_i)$

Micro Recall = $\sum_{i=1}^C TP_i$

$\sum_{i=1}^C (TP_i + FN_i)$

Micro F1 = $\frac{2 \times \text{Micro Precision} \times \text{Micro Recall}}{\text{Micro Precision} + \text{Micro Recall}}$

Worlds with discrete

feature: as key assumption is

Bayesian Decision Theory

for classification problem: supervised learning setting

(i) probabilistic model

(ii) look for decision rule that would give optimal Bayes Error (Lower bound on error)

which give notion of min error or min risk

example

Students

S1 S2 S3 ... SN : n samples in my data

each
Sample points
belongs to
one of C1 or C2 class

C1/C2

C1/C2

C1/C2

% available information

(pass or not pass)

hence supervised learning

Students

Known

- Probability of passing the course : We get to know for
- $P(C_1)$ & $P(C_2)$: Known - (it's fixed)

prior
probprior
prob

(A)

Known

- Choose whether to take the course or not

What should be the decision rule?

 $\Rightarrow$ Choose $C_1 : P(C_1) > P(C_2)$ $C_2 : P(C_2) > P(C_1)$ otherwisebased on
this

- What's the error we are gonna ~~get~~ make? : i.e. We choose C_1 but it was ideal to take C_2

Error prob : $P(\text{error}/s)$

$$= \min(P(C_1), P(C_2))$$

Why? (Think on it)

$$\text{ex. } C_1 - p_1 = 0.7$$

$$C_2 - p_2 = 0.3$$

$$\text{error} = 0.3$$

Date: / /

→ So we have features like background understanding

$S_i \mid - x_i$
 $\uparrow$
 feature

$P(u|c) \leftarrow$ class conditional probability density

$$\Rightarrow p(c|s) = p(c|u)$$

→ we want to know,
 $P(C_1 | m)$ or $P(C_2 | m)$

What is the class a

particular complex

Don't take?

This is the updated probability of the class

given the observed features

- It is what we are trying to predict/infer in a classification task

Classification

$$\rightarrow \underline{P(C_j, u)} = P(C_j | u) P(u) = P(u | C_j) \cdot P(C_j)$$

(The classification)

Bayes Formula

$$P(C_j | u) = \frac{P(u | C_j) P(C_j)}{P(u)}$$

current scenario
past scenario

Current Scenario

Scenario

with the highest
proximity in the predicted
class for the given set of
features)

This is known as posterior probability

⇒ How much ^{of} this class charges provided in

Where $P(n|c_j)$ is 'likelihood of c_j wrt n ' $\leftarrow$

probability of $\Rightarrow$ This means the class C_j for which $p(u|C_j)$

Observing the given

Is large is more likely to be true

evidence (features)

What is $p(u)$ $\leftarrow$ This is prob of obtaining the given evidence (features)

Given that the class
is true

$P(c_j)$? prior probability of c_j : This is the pb of a class before

$$\rightarrow p(x) = \sum p(x|c_j) \cdot p(c_j)$$

Considering any
features

NOTE: Likelihood is low. This is known as evidence

but the parameters of the model describe the outcome. This is generally not considered (the denominator part) as it is a normalising factor.

The model with the optimal

parameters maximize the probability.

(The probability is how probable an event or observation is for the given parameters of the model used)

sort of reassignment here

What should be my decision rule here?

$$\begin{cases} C_1 & \text{if } P(C_1|x) > P(C_2|x) \\ C_2 & \text{otherwise} \end{cases}$$

→ you'll give the label of class

→ is it a single value? prob distribution?

Generalization th

• S

• C

• Act

What is my error probability?

$$P(\text{error}|x) = \begin{cases} P(C_1|x) & \text{if } C_2 \text{ decided class} \\ P(C_2|x) & \text{if } C_1 \text{ decided class} \end{cases} = \min(P(C_1|x), P(C_2|x))$$

(This is known as Bayes error)

$$P(\text{error}) = \int_{-\infty}^{\infty} P(\text{error}, u) du$$

$$= \int_{-\infty}^{\infty} P(\text{error}|u) p(u) du$$

No decision ←

rule will give me probability less than this

Resulting in total error to be min

→ Training sample independent to each other

* → Decision rule based on prior property or posterior prob. The error associated with it gives us min value

Assumption:

Generalization th

- S
- C
- Act

Why? (Think)

loss

loss which has occurred when a decision has been taken (x_1) true class is

Avg over loss while taking decision (for a sample)

$$R = \int R(x(u)|u) p(u) du$$

Overall risk

- Assumption: Population densities known

Generalize the idea

- $S \vdash u \in \mathbb{R}^d$ ← set of features
- $C = \{c_1, c_2, \dots, c_k\}$: set of classes
- Actions / Decisions

$$\alpha_i = \{\alpha_1, \dots, \alpha_m\}$$

decision made

Why? (Think)

$$\ln(p(g|u), p(c_i|u))$$

- first it can be class i.e. $\alpha_1 = c_1$

Types of decision

$$\alpha_2 = c_2$$

- there is no decision made i.e. no class chosen

- loss FN: ← (previously we minimised the error so this is related with it)
- cost associated with each action

loss which has occurred when

a decision has been taken (α_i) provided true class is c_j

Note: Conditional loss

expectation

(This calculates the loss associated with a class provided the true)

- Conditional Risk: defined over loss

Avg over loss with taking decision

(for a sample)

$$R(\alpha_i | u) := \sum_{j=1}^k \lambda(\alpha_i | c_j) \cdot p(c_j | u)$$

expected loss associated with taking action α_i

here we want to minimize Conditional risk

$$R = \int R(\alpha(u) | u) p(u) du \Rightarrow \text{we want to take } \alpha_i \text{ with reduced}$$

Overall risk

$$\alpha_i^* = \text{argmin} (R(\alpha_i | u))$$

• Specific Example.

$$C = \{C_1, C_2\}$$

$$\alpha = \{ \overset{\alpha_1}{C_1}, \overset{\alpha_2}{C_2} \}$$

$$\lambda_{ij} := \lambda(\overset{\text{action}}{\alpha_i} | \underset{\text{actual class}}{C_j})$$

Conditional Risk

$$(i) R(\alpha_1 | u) = \lambda_{11} P(C_1 | u)$$

$$+ \lambda_{12} P(C_2 | u)$$

$$(ii) R(\alpha_2 | u) = \lambda_{21} P(C_1 | u) + \lambda_{22} P(C_2 | u)$$

{forgiven α

my decision

is α_1 which

is C_1 ?

What's my decision?

$$\Rightarrow \alpha_1 = C_1 : \text{if } R(\alpha_1 | u) < R(\alpha_2 | u)$$

$$\Rightarrow \lambda_{11} P(C_1 | u) + \lambda_{12} P(C_2 | u)$$

$$< \lambda_{21} P(C_1 | u) + \lambda_{22} P(C_2 | u)$$

Decision #1

$$\Rightarrow (\lambda_{21} - \lambda_{11}) P(C_1 | u) > (\lambda_{12} - \lambda_{22}) P(C_2 | u)$$

posterior

probabilities

is weighted by

loss/lost

What can we say about $\lambda_{21} - \lambda_{11}$ or $\lambda_{12} - \lambda_{22}$?

$\Rightarrow$ Are they positive or negative?

$\Rightarrow \lambda_{21} \rightarrow$ cost occurred when decision is C_2

Σ true label is C_1

$\lambda_{11} \rightarrow$ loss when decision is C_1

Σ true label is C_1

$$\lambda_{21} > \lambda_{11}$$

$\Rightarrow$ Both terms are positive (> 0)

$\Rightarrow$ (We just make the decision based on posterior probability, no need of conditional Risk)

Decision

(for error

Q

Q

Condi

Construct another classifier (provided previous one)
 → It calculates the below ratio:

Decision 2

$$\frac{P(x|c_1)}{P(x|c_2)} > \underbrace{\left(\frac{\lambda_{12} - \lambda_{11}}{\lambda_{21} - \lambda_{11}} \right) \frac{P(c_2)}{P(c_1)}}_{\text{Threshold}}$$

(for risk based error) ↑
 likelihood ratio

c_2 is true ⇒ The option is: we can build another classifier based on likelihood ratio. E.g. say the sample point belongs to a class if likelihood ratio > (some threshold)

Q Assignment 1: Derive D2 ~~from D1~~ (something)

Q Assignment 2:

$$\underbrace{\lambda(x_i | c_j)}_{\text{Zero-one-loss function}} = \begin{cases} 0 & i=j \\ 1 & i \neq j \end{cases} \begin{array}{l} \text{Assign 0 loss to a correct decision} \\ \text{Assign unit loss to an incorrect decision} \end{array}$$

Issue: All errors are equally costly } ← diff kind of incorrect decision happening (something)
 + (for eg. large and small if x is small, and we predicted large then also error is 1 & if it was predicted and then also when bias guard)

Summary

- (i) In probabilistic model (.....)
- (ii) 2 different decision rule

Conditional risk minimization

$$R(x_i | \bar{x}) = \sum_{j=1}^c \lambda(x_i | c_j) * P(c_j | \bar{x})$$

as and prediction shall have smaller diff error)

$$\Rightarrow R(x_i | \bar{x}) = \sum_{j \neq i} P(c_j | \bar{x}) = 1 - P(c_i | \bar{x})$$

(The very decision rule of Bayes' classifier automatically minimizes the conditional risk associated with an action.)

Bayesian Classifier What is like cont? discrete here?

Known N $\rightarrow S_i: x_i \in \mathbb{R}^d$: cont feature vector

$$\rightarrow p(x|c_j) \cdot p(c_j)$$

$$\text{min-error} \rightarrow C^* = \arg \max_{C_j} p(C_j|x)$$

$$\text{min-risk} \rightarrow \alpha^* = \arg \min_{\alpha_i} R(\alpha_i(x)|x)$$

$$\text{Binary Classification: } (\lambda_{21} - \lambda_{11})p(c_1|x) > (\lambda_{12} - \lambda_{22})p(c_2|x)$$

Choose C_1 if it's true (min risk)

The another form of this is:

$$\left(\frac{p(x|c_1)}{p(x|c_2)} \right) > \left(\frac{\lambda_{12} - \lambda_{21}}{\lambda_{21} - \lambda_{11}} \right) \frac{p(c_1)}{p(c_2)}$$

(likelihood ratio)

Another Classifier (Here obj is the classifier will be discriminant fn)
Known as Discriminant Function

Discriminant Function as classifiers

a) We are defining discriminant function $g_i(x)$ for each class C_i

b) Q:- The aim is to define appropriate discriminant function (How is this solved)

for this, the decision rule is defined:

$$C^* = \arg \max g_i(x)$$

To minimize $\xrightarrow{\text{Choice 1}}$ conditional $g_i(x) = -R(\alpha_i|x)$ risks

$\xrightarrow{\text{Choice 2}}$ $g_i(x) = p(C_j|x)$

$\uparrow$ min error rate

Few choices for discriminant function

choice 3 → Any monotonically increasing function of $g_i(x)$ in (1) & (2)

These can also be used

$$\begin{cases} g_i(x) = \frac{p(x|c_i) p(c_i)}{p(x)} \\ g_i(x) = p(x|c_i) p(c_i) \\ g_i(x) = \ln p(x|c_i) + \ln p(c_i) \end{cases}$$

Since logarithm is monotonically increasing function, it doesn't change the result

• for binary classifier, $g(x) = g_1(x) - g_2(x)$

$$g(x) > 0 \rightarrow c_1$$

$$g(x) \leq 0 \rightarrow c_2$$

c) if we have single/univariate normal density

$$p(x) = \frac{1}{(\sqrt{2\pi})\sigma} \exp\left(-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right)$$

$$p(x) \sim N(\mu, \sigma^2)$$

$$x \sim N(\mu, \sigma^2)$$

Mean: $\mu = E(x) = \int_{-\infty}^{\infty} x p(x) dx$

Variance: $\sigma^2 = E((x-\mu)^2) = \int_{-\infty}^{\infty} (x-\mu)^2 p(x) dx$

d) Our interest is to look at multivariate case
 ≠ Multivariate density (because x combination of dimensions vector)

multivariate normal density

$$p(x) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\left[-\frac{1}{2} (x-\mu)^T \Sigma^{-1} (x-\mu)\right]$$

$\uparrow$ $\uparrow$ $\uparrow$
 $1 \times d$ $d \times d$ $d \times 1$

$x \in \mathbb{R}^d$
 $\int x p(x) dx = \mu_j \in \mathbb{R}^d$
 $\Sigma \in \mathbb{R}^{d \times d}$
 $\int (x - \mu_j)(x - \mu_j)^T p(x) dx = \Sigma$

σ_{ii} : Variance, σ_{ij} : covariance
 $\sigma_{ij} = 0 \rightarrow x_i \& x_j$ are statistically independent
 $\sigma_{ij} = E((x_i - \mu_i)(x_j - \mu_j))$

Why cov should be $d \times d$ matrix?

$\rightarrow$ positive, semidefinite & symmetric

$|\Sigma| \leftarrow$ determinant of covariance matrix

$\Sigma^{-1} \leftarrow$ inverse of cov. matrix

What kind of classifier we will get?

Given: $S_i \mid \mu_i \in \mathbb{R}^d$

(Here we are assuming class conditional density follows a multivariate normal density function)

$p(x | c_i), p(c_i)$

• if we assume $p(x | c_i) \sim N(\mu_i, \Sigma_i)$

Note: $\mu_i \leftarrow$ mean vector of class c_i

$\Sigma_i \leftarrow$ covariance matrix of class c_i for i th class, the values (μ_i, Σ_i) are coming from the Normal class having μ_i & Σ_i

$\rightarrow$ (It defines a region in space st sample points go there)

• let's say discriminant functions,

$g_i(x) = \ln(p(x | c_i)) + \ln(p(c_i))$

$\Rightarrow g_i(x) = -\frac{1}{2} (x - \mu_i)^T \Sigma_i^{-1} (x - \mu_i)$

$-\frac{d}{2} \ln 2\pi$

(We have covariance matrix as diagonal matrix)

$-\frac{1}{2} \ln |\Sigma_i| + \ln p(c_i) \quad \text{--- (A)}$

Case I: $\Sigma_i = \sigma^2 I \leftarrow$ Identity matrix of appropriate dimension

• What does it tell the relationship among features?
 • Can anything be told about it?

Remarks

R1: (i) Σ_i is a diagonal matrix
 (ii) features are statistically independent, i.e. $\sigma_{ij} = 0$ when $i \neq j$
 $\rightarrow$ Cor both features x_i & x_j

(exp(A) & eqn are equal equivalent decision)

Since we are for discriminant which is defined

linear function

Since $x^T w$ all class depends

only independent

(iii) all features have the same variance σ^2

$$R2: |\Sigma_i| = (\sigma^2)^d$$

Σ_i same for all classification but in QDA (it's not)

$$R3: \Sigma_i^{-1} = \left(\frac{1}{\sigma^2}\right) I$$

$$\left(\frac{1}{\sigma^2} \left(-\frac{d}{2} \ln 2\pi - \frac{1}{2} \ln |\Sigma_i| \right) \right)$$

$g_i(x) = -\frac{1}{2\sigma^2} (x - \mu_i)^T (x - \mu_i) + \ln p(c_i)$
~~as it is~~ ~~hence to reduce computation~~ ~~same for all classes~~ ~~whatever terms not depend on x is not considered~~

With the particular $\Sigma_i = \sigma^2 I$, we want to look at exp (A),

$$g_i(x) = -\frac{1}{2\sigma^2} (x - \mu_i)^T (x - \mu_i) + \ln p(c_i)$$

$$= -\frac{1}{2\sigma^2} \left(x^T x - 2\mu_i^T x + \mu_i^T \mu_i \right) + \ln p(c_i)$$

(exp A) & the eqn are not equal but equivalent when making a decision

NOTE: our decision favors the prior if x is near the mean
 (equivalent in terms of decision rule)

same for all

$$= \frac{1}{2\sigma^2} \left(2\mu_i^T x - \mu_i^T \mu_i \right) + \ln p(c_i)$$

Since we are looking for discriminant function, which is defining regions

$$g_i(x) = w_i^T x + w_{i0}$$

linear discriminant function
 * (last page)

$$\text{where, } w_i = \frac{\mu_i}{\sigma^2}, w_{i0} = p$$

can be derived from here

Since $x^T x$ contribute equally/same in all classes hence removed so whatever is depended on (i) is considered

$$\begin{aligned} & \frac{\mu_i^T x + \ln p(c_i)}{2\sigma^2} \\ & \downarrow \text{special case} \\ & \Rightarrow w_i = \sum_{j=1}^d \mu_{ij} \\ & w_{i0} = \frac{1}{2} \sum_{j=1}^d \mu_{ij}^2 + \ln p(c_i) \end{aligned}$$

Averaging Strategies for Binary Metrics

NOTE: Micro Precision = Micro Recall = Micro F1 = Accuracy
in multi-class settings

(iii) Weighted Average ↙ accounts for class imbalance

- Calculate for each class
- take the average weighted by support (no. of true instances) for each class

$$\text{Weighted Avg} = \sum_{i=1}^C \left(\frac{S_i}{\sum_{j=1}^C S_j} \right) M_i$$

Multi Label Classification Metrics

- Example Based Metrics

(i) Exact Match Ratio (EMR) / Exact Accuracy

- fraction of samples where the predicted label set exactly matches the true label set

$$\text{EMR} = \frac{1}{N} \sum_{i=1}^N I(y_i = \hat{y}_i) \quad \begin{array}{l} \text{set of predicted labels} \\ \text{for instance } i \end{array}$$

↑
Total no. of instances

↑
set of true labels for instance i

- A prediction is only correct if all labels are perfectly predicted.

$I(\cdot)$: Indicator function (1 if true, 0 if false)

(ii) Hamming Loss

- Loss is better
- Avg fraction of labels that are incorrectly predicted per instance
- Measure the proportion of the instance label pairs that are misclassified

(0 is perfect)

- Sensitive to no. of labels

$$\begin{aligned} \text{Hamming Loss} &= \frac{1}{N} \frac{1}{K} \sum_{i=1}^N |y_i \Delta \hat{y}_i| \quad \begin{array}{l} \text{symmetric difference} \\ \text{(XOR operation on} \\ \text{the two sets, count} \\ \text{ing labels in} \\ \text{one set but} \\ \text{not the other)} \end{array} \\ &= \frac{1}{N} \frac{1}{K} \sum_{i=1}^N \sum_{j=1}^K I(y_i^j \neq \hat{y}_i^j) \end{aligned}$$

(III) Jaccard Index/Intersection over Union (IoU)

- Avg jaccard similarity coefficient betⁿ the true & predicted label sets over all instances

$$\text{Jaccard Index} = \frac{1}{N} \sum_{i=1}^N \frac{|Y_i \cap \hat{Y}_i|}{|Y_i \cup \hat{Y}_i|}$$

- Range: $[0, 1]$
- penalises both false positives & false negatives in the label sets

Label Based Metrics

- Adapt Binary Metrics by considering each label independently & then averaging
- let

$TP_j, FP_j, FN_j \leftarrow$ for j th label across all instances

(i) Macro Average: Treats all labels equally

$$\text{Macro Precision} = \frac{1}{L} \sum_{j=1}^L \frac{TP_j}{TP_j + FP_j}$$

Similarly,

$$\text{Macro Recall} = \frac{1}{L} \sum_{j=1}^L \frac{TP_j}{TP_j + FN_j}$$

$$\text{Macro F1} = \frac{1}{L} \sum_{j=1}^L \frac{2 \times \text{M.P.}_j \times \text{M.R.}_j}{\text{M.P.}_j + \text{M.R.}_j}$$

(ii) Micro Average: Averages each instance-label prediction pair

Favors performance
on common labels

equally

$$\text{Micro Precision} = \frac{\sum_{j=1}^L TP_j}{\sum_{j=1}^L (TP_j + FP_j)}$$

$$\text{Micro Recall} = \frac{\sum_{j=1}^L TP_j}{\sum_{j=1}^L (TP_j + FN_j)}$$

$$\text{Micro A} = \frac{2 \times \text{Micro Precision} \times \text{Micro Recall}}{\text{Micro Precision} + \text{Micro Recall}}$$

11.10 Weighted Average

(iv) Sample Average

- calculate metrics for each instance individually, then average

(Consider the performance on each sample individually)

$$\text{Sample Precision} = \frac{1}{N} \sum_{i=1}^N \frac{|y_i \cap \hat{y}_i|}{|\hat{y}_i|}$$

$$\text{Sample Recall} = \frac{1}{N} \sum_{i=1}^N \frac{|y_i \cap \hat{y}_i|}{|y_i|}$$

- Handle division by 0 if $|y_i| = 0$ or $|\hat{y}_i| = 0$

→ Mostly considered 1

- based on F1,

$$\text{Sample F1} = \frac{1}{N} \sum_{i=1}^N F1_i$$

Correlation & Aggregation Metrics

- measure the overall agreement betⁿ predictions & true values.
- Sometimes offers advantages on imbalanced datasets

(1) Pearson Correlation Coefficient (r)

- measure linear correlation between two continuous variables
- less common as a direct classification metric
- Compare predicted probabilities to binary outcomes or feature analysis

for two variables X & Y with N paired samples (x_i, y_i) :

$$r = \frac{\sum_{i=1}^N (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^N (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^N (y_i - \bar{y})^2}} = \frac{\text{Cov}(X, Y)}{\sigma_x \sigma_y}$$

$$\sigma_x = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2}$$

where: $\bar{x}, \bar{y} \rightarrow$ Sampled mean

$\sigma_x, \sigma_y \rightarrow$ Sampled standard deviations

Range $\rightarrow [-1, +1]$

↑ perfect +ve linear correlation
↓ perfect -ve linear "

0: No linear correlation

(ii)

- A
- reg
- the c

AP: Actual

P: Predicted

$$TP = \sum A_i P_i$$

$$TN = \sum (1 - A_i)(1 - P_i)$$

$$FP = \sum (1 - A_i) P_i$$

$$FN = \sum A_i (1 - P_i)$$

$$A_0 = \frac{TP + FN}{N}$$

$$P_0 = \frac{TP + FP}{N}$$

$$\sigma_A = \sqrt{\frac{1}{N} \sum_{i=1}^N (A_i - A_0)^2}$$

$$\sigma_P = \sqrt{\frac{1}{N} \sum_{i=1}^N (P_i - P_0)^2}$$

$$\sigma_P^2 = P_0(1 - P_0)$$

NOT

Agreement: $TP \times TN$
 Disagreement: $FP \times FN$

(ii) Matthews Correlation Coefficient (MCC)

- A correlation ^{coefficient} betⁿ the observed & predicted binary classification
- regarded as a balanced measure which can be used even if the classes are of very different sizes

A_i : Actual

P_i : Predicted

$$MCC = \frac{(TP \times TN) - (FP \times FN)}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

$$TP = \sum A_i P_i$$

$$TN = \sum (1 - A_i)(1 - P_i)$$

$$FP = \sum (1 - A_i)P_i \quad \text{Range: } [-1, 1]$$

$$FN = \sum A_i(1 - P_i) \rightarrow +1 \text{ implies a perfect prediction}$$

$$\rightarrow 0 \text{ " no better than random prediction}$$

$$A_0 = \frac{TP + FN}{N} \rightarrow -1 \text{ " total disagreement betⁿ prediction & observation}$$

$$P_0 = \frac{TP + FP}{N} \text{ High values req both positive & negative classes to perform good}$$

$$\sigma_A = \sqrt{\frac{1}{N} \sum_{i=1}^N (A_i - A_0)^2} \rightarrow A_0(1 - A_0) = \sigma_A^2$$

Advantages:

$$\sigma_P = \sqrt{\frac{1}{N} \sum_{i=1}^N (P_i - P_0)^2} \text{ (i) preferred over accuracy & F1 score}$$

(ii) because it requires the model to correctly classify both the majority & minority class to achieve a high score

$\Downarrow$

$$\sigma_P^2 = P_0(1 - P_0) \text{ (iii) if model always predict majority class } \Rightarrow MCC \rightarrow 0 \text{ but accuracy is high}$$

(iv) Similarly MCC avoids pitfalls of F1 where high scores can sometimes be achieved by models poor at predicting one class if the other is predicted exceptionally well.

NOTE: Bias Variance Tradeoff Says

(i) Increasing model complexity generally $\downarrow$ bias

$\uparrow$ variance

(ii) decreasing " " " $\uparrow$ bias

$\Downarrow$ variance

$$\sum_{i=1}^N (x_i - \bar{x})^2$$

$$\# \text{ Precision} = \frac{TP}{TP+FP}$$

- Use case: when cost of FP is high

ex. spam detection

Search engine results (don't want unwanted results top)

- Limitations: ignore false negatives

→ predict positive only when confident → missing actual

positive instances but high precision

- Exploitation: perfect precision by predicting only 1 instance

as positive, provided it's correct

$$\# \text{ Recall} = \frac{TP}{TP+FN}$$

- Use case: when cost of FN is high

ex. medical diagnosis for serious diseases (don't want to

miss person with disease)

fraud detection (don't want to miss fraudulent transactions)

- Limitations: ignore FP

→ perfect recall by predicting everything pos

→ but this may lead to many FP & low precision

- Exploitation: perfect recall by predicting everything as positive

$$\rightarrow (1 - FPR)$$

$$\# \text{ Specificity (TNR)} = \frac{TN}{TN+FP}$$

Ability to identify negative class

- Use case: when correctly identifying negative is crucial

ex. drug testing (no false accusation of drug use)

- Exploitation: ignore positive class performance (TP, FN).

Negative Predictive Value (NPV)

(measures accuracy of negative predictions)

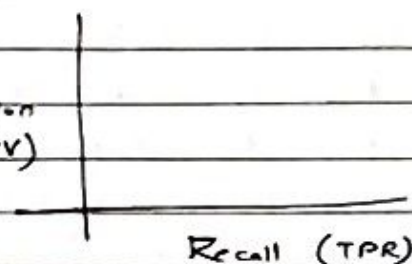
$$NPV = \frac{TN}{TN + FN}$$

- use case: When confidence in a negative prediction is needed
ex. if a diagnostic test predicts a patient is healthy, NPV tells us the prob they are actually healthy

- Limitations: can be high even if the model misses many positive cases (high FN), especially if the negative class is large

PR Curve

- As threshold decreases, more instances classified as positive increases recall but decreasing Precision



- Ideal Model: curve goes from (0,0) & then straight across to (1,1)
- Random Model: A horizontal line at the level of positive class prevalence (proportion of positive ~~class~~ instances in the dataset)

$$P = \frac{TP}{TP + FP} \approx \frac{Pos}{Pos + neg} = \text{Prevalence}$$

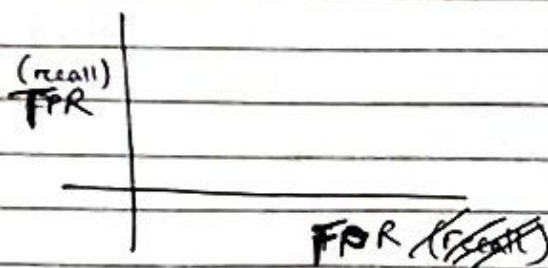
- Good model: curve towards the top right corner
- AUC-PR: More informative than AUC-ROC for highly imbalanced datasets where the large no. of true negatives can inflate AUC-ROC

(our primary interest is positive class)

(area $\rightarrow 1$)

ROC curve

- lower threshold $\Rightarrow \uparrow FPR$ & $\uparrow TPR$



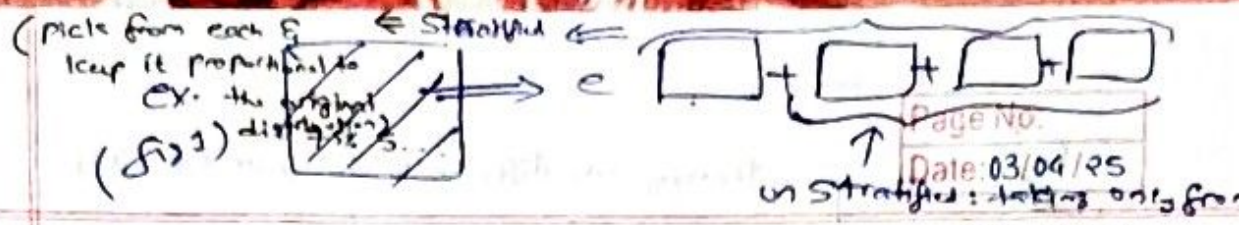
- Ideal Model: Curve goes straight up to $(0,1)$ then goes straight across $(1,1)$. Top left corner is best
 - Random Model: Diagonal line from $(0,0)$ to $(1,1) \Rightarrow \text{TAR} = \text{FPR}$
 - Good Model: Bows out towards the top left corner
 - AUC-ROC: Area under ROC curve
- $\rightarrow$ ranges from 0.5 to 1.0 (perfect)
(random)

$\rightarrow$ represents the probability that the model ranks a randomly chosen positive instance higher than a randomly chosen negative instance. Less sensitive to class imbalance than AUC-PR

NOTE!!

(Previously we evaluated the performance of model on training set / how well it is performing on the given set, $\rightarrow$)

but here we check on unseen data)



- a) How well a classifier do on unseen data?
- model overstrain
 - distribution shift : if the test data is very different
 - What can we do to measure how the model will perform

on an unseen data?

Method 1:

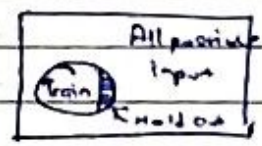
Lazy eval : (no estimation of performance on unseen data done) $\leftarrow$ (CRM)

- this is known as Reusability
- only report on the available data
 - full data used
 - overestimating performance
 - So test is always done on the same data on which it is trained

Method 2: (1-Shot)

Keep some part of the data from training

$\rightarrow$ check performance on holdout set



Train 1 model $\rightarrow$ (A) $\rightarrow$ If $L_{tr} \approx L_m$ found during training, it might give high for some specific labels in case of test (not good) $\leftarrow$ unstratified

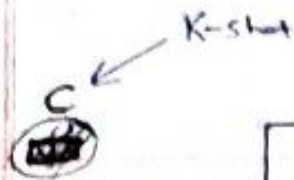
$\rightarrow$ here go for stratified labels (B) : maintain ratio of labels in training dataset (REFER fig 1)

(#1-shot) NOTE: we can have multiple models on the same training set by changing the hyperparameters etc; Check whichever performance is good $\Rightarrow$ Select & deploy it whichever performs good on test set

Why? because when selecting model we didn't consider the holdout set so avoiding leakage of data but this is not good as leakage of data

- Train model $\leftarrow$ train set
- Variation of model $\leftarrow$ validation set $\leftarrow$ out of M models chosen 1
- Test on holdout set $\leftarrow$ Test set

Assumption $\rightarrow$ gives the estimated performance test set (the holdout) is similar to real world unseen data



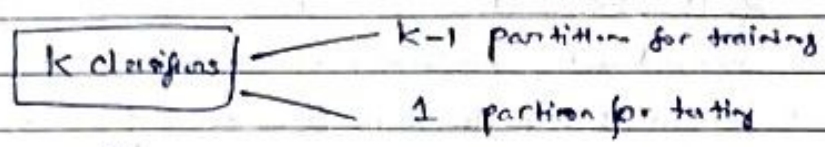
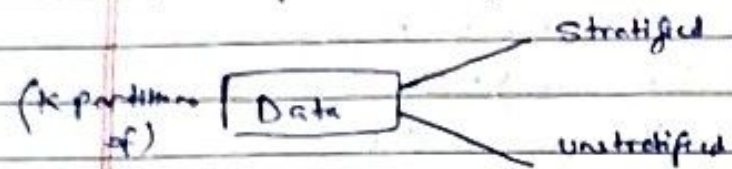
training sets different & testing

b) K-Fold Cross Validation

• We create K testing set instead of 1 hold out set to check performance

• $K=N$: Leave one out method
 → each point used for testing only once

every point is possible to be in test set



Test Train
 $M_1: P_1 \quad P_2 \dots P_K$
 $M_2: P_2 \quad P_1 P_3 \dots P_K$

but? $\otimes \rightarrow$ avg? $\otimes \rightarrow$

how to check which but among is

if we want to pick 1/K we compute same metric (precision, accuracy etc)

- K model versions
 - Compute mean
 - risk performance variation
- for each model

helps in determining

what does it tell P.

what performance I can get on unseen data

We compute confidence intervals

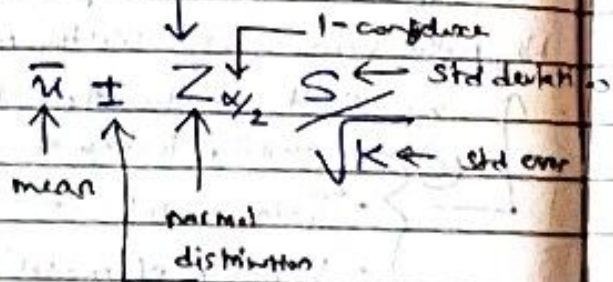
higher confidence interval
 ⇒ go further away mean

This gives sort of raw score

• While deploying.

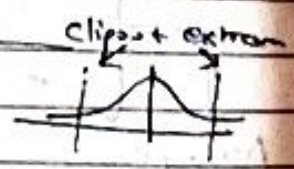
we train on whole data & not K different version

• Considered as normal distribution



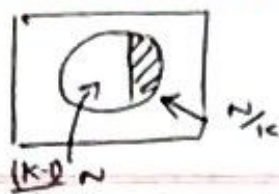
• This makes us to ask how much area is covered

• NOTE: the reason was I can compute M (precision, accuracy etc) on all K , & take avg but not good



• So we go for finding confidence (but on what?)

For:



but now $\rightarrow$ Training size up to N but we need test data as well. So we pick the training sample randomly from existing data (refer fig 2)

(D)

Training size in previous one was one less but to maintain training data size $\Rightarrow$ Bootstrapping

K-fold

Maintain train data size

• Something with sampling with replacement done: for creating a dataset of size N

• probability of getting not selected for $E_0 = \left(1 - \frac{1}{N}\right)^N \approx 0.36$

(rounds of a data point)

$\rightarrow$ The unselected points become the test set (i.e. out of bag data)

E_0 : Out of bag data very small $\therefore$ so performance evaluation on it & reporting not efficient

• points not selected for training data

avg. variance?

$$E_{0.64} = 0.36 (\text{Training performance}) + 0.64 (E_0)$$

$\rightarrow$ final estimated performance

hence go for confidence intervals



Fig 2

Note:

K-fold $\rightarrow$

no hyperparameter selection

(only model performance evaluation)

Here one of the points selected by sampling with replacement this is bootstrapping

Note: Bootstrapping is different, we don't consider the testing set over here, the performance is estimated based on above formula

- a) K-NN Classifier → data is kept as train & test
 → 2 methods: (1) train on train data, test on test data
 (2) cross-validation
 - no training of classifier
 - for test, we find top K nearest neighbour & give the label
 Adv: no computation lost

Disadv: Compute distance with every point in the data set during testing (store prediction) - n distances
 for smaller dataset: (error 0) ← k=1 & train=test
 ⇒ (low bias, high variance)

- Change of data: no issues
- Change of label: no issues
- Privacy concern: problem - as training data gets exposed
- Optimisation: n distances req (currently)

We are interested in just K distances

ex. KD trees (done by)

→ Prediction: $O(\log n)$ → In normal world

b) Evaluation Technique (contd)

	Resampling	Bias	How much the training data is deprived of?
Hold Out (fast)	X	High	Var
K-fold (slow)	Not repeated	Moderate	mod
Bootstrap (slowest)	Repetition	Higher (E_0) ↓ Moderate ($E_0 + 0.52$)	low

→ Bias of CV & Bootstrapping

- c) Repeated CV: previously CV was done only 1 time
 → start with fresh set of data: hence no repetition

give
performance
estimation

→ large no. of trials

How?

CV1:

P1	P2	P3	P4
----	----	----	----

CV2:

--	--	--	--

← 4 iterations of model

the previous point which belonged to P1 now can go to either of P2, P3, P4 for CV2

⇒ for each CV, we get a performance score

⇒ compute average → we get a stable performance estimate

1: (1) treat them as 1 set & get the avg (only)
(Hyperparameter tuning)
test: first k-NN
get more insight

gives ~~both~~ the hyperparameters

~~no~~ best on performance

Best hyperparameters

(by doing something)

d) Hyperparameter Tuning

- let's do 3 fold CV $\rightarrow$ folds should be constant here

$\lambda: 0.2 \quad 0.7$

CV1: $\lambda = 0.2 \rightarrow$

CV2: $\lambda = 0.7 \rightarrow$

m_1, m_2, m_3

m_1, m_2, m_3

(6 models, 6 performance estimates)

estimate get

final estimate

Average

Average

for hyperparameter

estimate get

final estimate

Average

Average

which ones

is best go

with it

- Model with 4 hyperparameters config

but not good

(overestimation happens)

5 fold CV

$$\# \text{ models} = 5 \times 4 + 1$$

for deciding

the hyperparameters

for training the final model

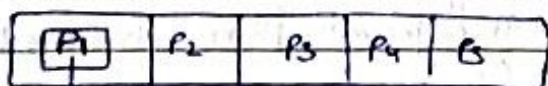
on whole data set

$\rightarrow$ how many times do we run the model here? (5 times?)

Variance of Nested CV: Done for hyperparameter tuning + performance estimation

- Outer CV $\rightarrow$ Inner CV {basic structure}

Outer CV

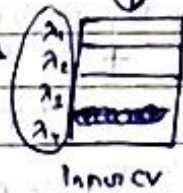


ex. 5 fold outer

3 fold inner

4 hyperparameters

(for each λ we run the inner CV)



Inner CV

$$\# \text{ models} = 5 + (3 \times 4) \times 5 + (12) + 1$$

for outer

hyperparameters

$M_1: \text{Test: } P_1, \text{ Train: } P_2, P_3, P_4, P_5$

for one particular setting of

λ (i.e. λ_1)

Make 3 partitions of these

Create $M_1': \text{Test: } D_1, \text{ Train: } D_2, D_3$

$M_1'': \text{Test: } D_2, \text{ Train: } D_1, D_3$

$M_1''': \text{Test: } D_3, \text{ Train: } D_1, D_2$

repeat for

$\lambda_1 (i \neq j)$

get the avg

value E_j

then go for next fold

i.e. $P_i (i \neq j)$

Report λ for

P_i

- Estimation of performance done on different data

Set other than the training set

here it's an advantage

Previously 1 fold: It was overfitting

- nested CV $\rightarrow$ more reliable estimation of model performance

$$\# \text{ models: } 5 + 60 + \textcircled{12} + 1$$

hyperparameter setting

e) Which model to select?

Model A: 2 hyperparameter configurations

" B: 3 " "

used for deciding performance

Simple CV with 5 fold:

$$A: 5 \times 2 + 1 = 11$$

$$B: 5 \times 3 + 1 = 16$$

(We need to select one among them)

$\Rightarrow$ Which gives an idea on which

Do we have all 4 hyperparameters [setting] is better model before only? (Yes)

Nested CV - for ~~deciding~~ ~~hyper~~ estimating performance

$\rightarrow$ consists of outer CV & inner CV

+ to find out which hyperparameter

Class example:

P_1	P_2	P_3	P_4	P_5
-------	-------	-------	-------	-------

$\rightarrow$ i) Data partitioned into 5 sets

Now since each CV_i will

run an inner CV of 3-fold

with 4 different hyperparameter settings

ii) because we go for 5 fold outer CV

$\Downarrow$ out of

to determine 4

hyperparameters

Let's say: We are doing CV,

Test set: P_1

Train set: $P_2 + P_3 + P_4 + P_5$

~~the CV~~

~~CV~~

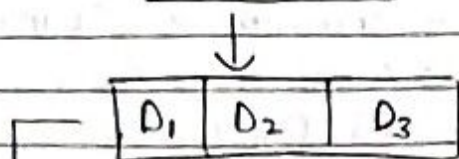
~~CV~~

~~CV~~

for CV

What we do is?

⇒ we divide $[P_2 + P_3 + P_4 + P_5]$ into 3 partitions as 3 fold inner CV



run this using $\lambda_1, \lambda_2, \lambda_3, \lambda_4$

This gives us 3×4 models

→ as a good hyperparameter setting

next with that hyperparameter

setting, we train a new model

on $[P_2 + P_3 + P_4 + P_5]$

this new trained model

is tested on P_1

which gives us the performance

for each λ , it will

be repeated 3 times

Test	Train
D_1	D_2, D_3
D_2	D_1, D_3
D_3	D_1, D_2

now this is repeated for the remaining possible partitions out of the 5 → 4 then take average

Hence we get: $(\frac{3 \times 4}{1} + 1) \times 5$

$(2 \times 3 + 1) \times 4$
= 60

for hyper parameters

final

for ~~performance~~

model on the

best hyperparameter

(given in Q)

now, $(3 \times 4 + 1)$ has extra:

→ so far we gave us performance estimation

→ how to select final hyperparameters & get the model

we do 3 fold

final hyperparameter selection

Which is best?

Initial partition: $[P_1 | P_2 | P_3 | P_4 | P_5]$

⇒ $[D_1 | D_2 | D_3]$

⇒ 3 fold CV on 4 different

train on full data set

← best λ

← λ setting

⇒ gives the final trained model

→ clean data need of the time

→ Hitesh Arora - Amazon

→ date lasting etc (imp)

→ different stakeholders

→ (i) Model - in research & production (diff)

(ii) Amazon Scout - requirements (LIDAR, camera)

(iii) LLM Model - Agentic workflows

• Agents existed from beginning

• Use of Agents in LLM

→ Not perfect model, it's reliability

• like e we don't focus on accuracy much (but in research & college, yes)

→ Amazon Scout (used transformers (check))

→ Development with LLM model (Try)

→ Deep learning, V/s Rn V/s LLM V/s Where does each hold?

→ can do classification as

→ Artificial General Intelligence?

→ ~~AGI~~ P (or AGI is) Intelligence

→ Mental model on code base - can AI able to give everything : (Code Reviews still done)

→ So reading code is still required for development

→ Hence having the skill is (must)

→ Deep learning resources? → (i) Fundamental of models

→ Prof, Courses online

(ii) For new models → Papers, AI tools (!!), to understand

Longche Landgrift (notes)

(iii) Projects (Small ones)

→ The crux of it is using probability to make classification decisions

Bayesian Classifier

Min Error Classification

min Risk

Decision Rule

$$\arg \max P(C_i | x)$$

$$\arg \min R(x_i | x)$$

$$P(C_i | x) \propto P(x | C_i) P(C_i)$$

This sort of classifier is

• Class conditional

usually either given

or can be estimated

Why is it called that?

⇒ Since data points can also be generated

← { called generative classifier

{ Generative, Discriminative classifier }

Discrimination

• not just classifier
for sample
• but data points/
density can be
computed

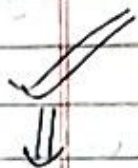
⇒ We can get another classifier based on

discriminant function = let's say $g_i(x) : D \rightarrow F$

Decision rule: $\arg \max g_i(x)$

⇒ Binary Classification: $g(x) = g_1(x) - g_2(x)$

⇒ We can get discriminant function in various ways, one of the ways is:



$$\text{if } \rightarrow g_i(x) = \ln P(x | C_i) + \ln P(C_i)$$

Derive it if $\rightarrow P(x | C_i) \sim N(\mu_i, \Sigma_i)$

like for

each class

$$g_i(x) = -\frac{1}{2} (x - \mu_i)^T \Sigma_i^{-1} (x - \mu_i)$$

$$- \frac{d}{2} \ln 2\pi - \frac{1}{2} \ln |\Sigma_i|$$

(features have same variance & linearly independent)

Case I: $\Sigma_i = \sigma^2 I \forall i$

Each of these belong to a region/class

This is derived from the probabilistic classifier but simpler

$$g_i(x) = w_i^T x + w_{i0}$$

— linear classifier (or discriminant function)

$$\rightarrow w_i = \frac{\mu_i}{\sigma^2}$$

$$\rightarrow w_{i0} = \frac{-1}{\sigma^2} w_i^T \mu_i + \ln P(c_i)$$

aka Gaussian discriminant classifier

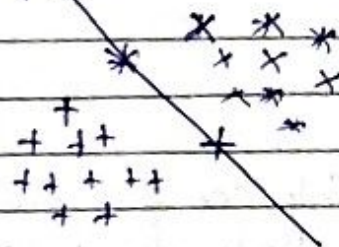
(why?.....)

→ Same as separating hyperplane

Decision Boundary

$$\rightarrow g_i(x) = g_j(x)$$

$$w_i^T x + w_{i0} = w_j^T x + w_{j0}$$



Algebraic manipulation

It's a

separating hyperplane

passing through

$\mu_0 \in$

normal to

w

$$\leftarrow w^T (x - \mu_0) = 0$$

NOTE: vectors are

considered as

$$x \in \mathbb{R}^{d \times 1}$$

hyperplane

Where $w = \mu_i - \mu_j$: hyperplane separating the two regions R_i & R_j

$$\mu_0 = \frac{1}{2} (\mu_i + \mu_j) - \frac{\sigma^2}{\|\mu_i - \mu_j\|^2} [\ln \frac{P(c_i)}{P(c_j)} (\mu_i - \mu_j)]$$

Special case#

Case 1a: If $P(c_i) = P(c_j)$, the separating hyperplane would be the bisector of means

(??)

→ μ_0 : midpt of line joining μ_i & μ_j

→ separating hyperplane is $\perp^r$ bisector of the line joining means

Case 1b: $P(c_i) \neq P(c_j)$

(feature mean class mean)

$\Rightarrow$ The separating hyperplane is shifting only a

Assumption $\Rightarrow$ To either of the class

Interpret separating hyperplane

class conditional density $N(\mu_i, \sigma_i^2)$

Which class? like if $P(c_i) < P(c_j)$ or $P(c_i) > P(c_j)$

Case 1c: $P(c_i) = P(c_j) \quad \forall i, j$

like if more than 2 classes $\Rightarrow$ they are equiprobable

Verify: $g_i(m) \propto -\|m - \mu_i\|^2$

$\Rightarrow$ To Do:

Same for all classes

Case II: $\sum_i = \sum$

$\begin{bmatrix} * & * & * & 0 & 0 & * \\ * & * & & & & \end{bmatrix}$

(i) Write $g_i(m) = ?$ down.

(ii) What happens $P(c_i) = P(c_j) \quad \forall i, j$

(iii) Do we get linear discriminant function?

(iv) What is the form of decision boundary / separating hyperplane?

Case III: $\sum_i =$ arbitrary different for each class

This is of the quadratic discriminant form - (Verify it)

Previous cases were continuous, but now, what if features are discrete?

x_i : taking one of 1 to m_i discrete values

Assumptions:

Sp1 Case: • Binary features - $x_i \in \{0, 1\}$
• Features are conditionally independent

$\Rightarrow$ it is a binary classifier

$P_1 = P(x_1=1 | c_1)$

$P_2 = P(x_1=1 | c_2)$

$$P(c_i | x) \propto p(x | c_i) P(c_i)$$

Page No:

Date: / /

features are conditionally independent given the class (assumption)

$$p(x_1, \dots, x_d | c_1) = \prod_{i=1}^d p(x_i | c_1)$$

$$= \prod_{i=1}^d p_i^{x_i} (1-p_i)^{1-x_i}$$

if $x_i = 1$, $(1-p_i)$ goes away

$$p(x_1, \dots, x_d | c_2) = \prod_{i=1}^d q(x_i | c_2)$$

$$= \prod_{i=1}^d q_i^{x_i} (1-q_i)^{1-x_i}$$

$$\Rightarrow g(x) = \ln \frac{P(x | c_1)}{P(x | c_2)} + \ln \frac{P(c_1)}{P(c_2)}$$

in this particular case,

$$\text{verify: } g(x) = \sum_{i=1}^d w_i x_i + w_0$$

$$\begin{cases} \rightarrow g(x) > 0 \rightarrow \text{class 1} \\ \rightarrow g(x) < 0 \rightarrow \text{class 2} \end{cases}$$

where,

$$\text{verify} \leftarrow \begin{cases} w_i = \ln \left(\frac{p_i (1-q_i)}{q_i (1-p_i)} \right) \end{cases}$$

$$w_0 = \sum \ln \left(\frac{1-p_i}{1-q_i} \right) + \ln \left(\frac{P(c_1)}{P(c_2)} \right)$$

How do we objectively compare different regions?
like Bangalore & San Francisco

Step 1# Embedding: capture semantic/structural similarity
Creation of ^{numerical} vectors using objects/words
↑
Created by word2vec
→ Why? — because can't be represented in form of numerical or categorical data
words are related with close ones (something)

Step 2 How do we represent a region? of words
Why do we want to do it? — like finding a region where cost is high/low, where to install next McDonalds store?
→ What sorts of feature will you take?
ex. proximity to the places, demographics, nice water body or park
Dataset

how to convert these for ML models can understand? → embeddings
→ for discussion: 2 datasets considered

Static data ← { (i) Mobility Dataset
(ii) Satellite Images — Similar to facial embeddings
data, how is it handled?

Correlation, Causation, Fallacy

Causal ML

↓
doesn't tell everything

is all about directionality

→ If A & B are correlated, which causes which?

a) could be: $A \rightarrow B$

b) or: $B \rightarrow A$

c) or: causes both A & B

Simpson's Paradox

How do we know to stop or these are enough features for a model?

- Randomized control test

→ something like we test one group with something & others with another & check the output

not good for cases like to detect if smoking causes cancer

→ every person has 2 potential outcomes

Causal Model - Potential Model

Y_1 : if taken (.)

Y_0 : if not taken

Potential outcomes independent of the assignment (Smtg)

Frick-Waugh Level Theorem - a way to isolate the effect of one variable in a multiple

Summary

1) Correlation $\neq$ Causation → one variable directly affects the other

↓
• 2 var are correlated if they move together.

• It shows association but not cause

2) ML models often learn patterns (correlations) but that's not enough when we want to intervene or understand why something happens

3) ~~If~~ Simpson's Paradox

- A trend appears in several subgroups of data but reverses when the groups are combined.

CX.	Group	Treatment Success	Control Success
	Men	90%	80%
	Women	70%	60%
	Combined	75%	85%

- Why? $\rightarrow$ Maybe more Women (with lower success) are in treatment group overall. (confounders)
- Always consider hidden variables when analysing data.

4) Randomised Controlled Trials

- A way to test causality by randomly assigning subjects to: (i) Treatment group

(ii) Control group

Why randomisation? - remove bias & confounding

We can only observe one of them

at a time

linear regression

$$\left\{ \begin{pmatrix} (1) & (2) \end{pmatrix} \dots \begin{pmatrix} (1) & (2) \end{pmatrix} \right\} = 1$$

$$\{ \dots \} = 1$$

$$\underset{\uparrow}{\operatorname{argmax}} \frac{p(c_j | n)}{p(n | c_j) p(c_j)}$$

To find this: $p(n | c_j) \sim \underline{f(n, \theta)}$

$$N(\underbrace{\mu_j}_{\text{known}}, \underbrace{\Sigma_j}_{\text{known}})$$

Notes

Let's consider: $p(n | c_j) \sim N(\mu_j, \Sigma_j)$

What if μ_j unknown & Σ_j known
if μ_j & Σ_j unknown

$$p(c_j) \sim \text{unknown}$$

Q How do you estimate the parameters?

Parameter Estimation

- (a) Maximum Likelihood Estimation (MLE)
- (b) Bayesian Estimation

(our course is focused on this)

how does both differ?

(i) MLE assumes parameter is fixed

(ii) Bayesian estimator considers it as random variable with known prior

→ Try to find a parameter value which gives maximum probability for the observed/given data

Observation/data leads to posterior density/probability

Maximum Likelihood Estimator

• General setting

$$D = \left\{ (\overset{\text{index of sample}}{x^{(i)}, y^{(i)}}), \dots, (x^{(n)}, y^{(n)}) \right\}$$

$$y^i \in \{c_1, c_2, \dots, c_K\}$$

• How do we use MLE? Sample belonging to class 1

→ data divided into k subsets: $D_1, D_2, \dots, D_k$

→ samples in D_j are i.i.d (independent & identically distributed) — (P?)

→ $P(y|C_j)$ is assumed to be coming from parametric form $(\sim f(\underset{\substack{\text{particular sample} \\ \text{particular} \\ \text{belonging to class}}}{x}, \underset{\substack{\text{random variable}}}{\theta_j})$

→ θ_i, θ_j : functionally independent $[i \neq j]$

parameters ← If this is known we can have our classifier computed by this way

• What is the likelihood function?

$L(\theta_j | D_j)$ in general (on), not considered.
↖ index j
 Parameter θ_j (for) D_j $L(\theta_j, D_j) := P(D_j | \theta_j)$
 subset joint probability

(doubtful) distribution $= P(x_{D_j}^{(1)}, x_{D_j}^{(2)}, \dots, x_{D_j}^{(n_j)} | \theta_j)$
→ since it is i.i.d, therefore
 $= \prod_{i=1}^{n_j} P(x_{D_j}^{(i)} | \theta_j)$

AIM: estimate the parameters

θ_j of each class conditional densities independently

$L(\theta, D) = \prod_{i=1}^n P(x^{(i)} | \theta)$
 In general we

Say this but for each subset we define

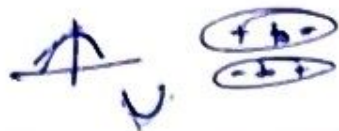
$$\hat{\theta}_j = \underset{\theta_j}{\operatorname{argmax}} \prod_{i=1}^{n_j} P(x_{D_j}^{(i)} | \theta_j)$$

• Generally we consider log likelihood

$$l(\cdot) = \sum_{i=1}^{n_j} \ln P(x_{D_j}^{(i)} | \theta_j)$$

$$\rightarrow \hat{\theta}_j = \underset{\theta_j}{\operatorname{argmax}} \left(\prod_{i=1}^{n_j} P(x_{D_j}^{(i)} | \theta_j) \right) = \underset{\theta_j}{\operatorname{argmax}} \sum_{i=1}^{n_j} \ln P(x_{D_j}^{(i)} | \theta_j)$$

ToDo: Just check how it came



• Univariate Normal Distribution

$$P(x_k | \underbrace{\mu, \sigma^2}_{\text{unknown}}) = \frac{1}{(\sqrt{2\pi})\sigma} \exp\left[-\frac{1}{2} \left(\frac{x_k - \mu}{\sigma}\right)^2\right]$$

That's why

$$\ln(P(x_k | \mu, \sigma^2)) = -\ln(\underbrace{(2\pi)^{1/2} \sigma}_{(2\pi\sigma^2)^{1/2}}) - \frac{1}{2} \left(\frac{x_k - \mu}{\sigma}\right)^2$$

$$\ell(\mu, \sigma^2) = \sum_k \left(-\frac{1}{2} \ln(2\pi\sigma^2) - \frac{1}{2\sigma^2} (x_k - \mu)^2 \right)$$

$$\frac{\partial \ell}{\partial \mu} = 0 \quad ; \quad \frac{\partial \ell}{\partial \sigma^2} = 0$$

Verify: $\hat{\mu} = \frac{1}{n} \sum_k x_k$ ← Sampled mean

$$\hat{\sigma}^2 = \frac{1}{n} \sum_k (x_k - \hat{\mu})^2$$
 ← Sampled variance

Exercise (for next class)

for posterior pb $\rightarrow P(c_j | x) = P(x | c_j) P(c_j)$ — (A)

$$P(x | c_j) \sim N(\mu_j, \Sigma_j)$$

multivariate normal distⁿ

Step 1:

req μ_j calculation { Case 1: μ_j — unknown
 $\Sigma_j = \Sigma$ μ_j known

both μ_j & Σ_j { Case 2: μ_j, Σ_j — unknown

$$P(y=1) = q, \quad P(y=0) = 1 - q$$

$$\hat{q}$$

$\Rightarrow$ Then we can find (A)

$$D = \{(x^{(1)}, y^{(1)})\}, \dots, (x^{(n)}, y^{(n)})\}$$

$$x^{(i)} \in \mathbb{R}^d, y^{(i)} \in \{0, 1\}$$

of parameters
 $(d^2 + d + 1) \times 2$
 from Σ from μ_0, μ_1

Assumptions:

$$(i) p(x|y=0) \sim N(\mu_0, \Sigma)$$

Unknown

$$(ii) p(x|y=1) \sim N(\mu_1, \Sigma)$$

unknown

$$(iii) p(y=1) = q \leftarrow \text{unknown } q \in \mathbb{R}$$

$$p(y=0) = 1 - q$$

$\mu_0 \in \mathbb{R}^d$	$\Sigma \in \mathbb{R}^{d \times d}$
$\mu_1 \in \mathbb{R}^d$	q

How to use Maximum Likelihood concept to find the parameters
 $\Theta = \{\mu_0, \mu_1, \Sigma, q\}$

Likelihood

$$L(\Theta|D) = p(D, \Theta)$$

$$= p((x^{(1)}, y^{(1)}), \dots, (x^{(n)}, y^{(n)}), \Theta)$$

$$= \prod_{i=1}^n p(x^{(i)}, y^{(i)}, \Theta)$$

because of
 Independence

$$= \prod_{i=1}^n p(x^{(i)} | y^{(i)}, \Theta) p(y^{(i)}, \Theta)$$

To make it

Simpler, take log-likelihood $\because$ As it is non decreasing function
 $\Rightarrow$ maximizing $l(\Theta, D) = L(\Theta, D)$

$$l(\Theta|D) = \sum_{i=1}^n \ln p(x^{(i)} | y^{(i)}, \mu_0, \mu_1, \Sigma, q)$$

$$+ \sum_{i=1}^n \ln p(y^{(i)}, \mu_0, \mu_1, \Sigma, q)$$

$$\Theta^* = \arg \max_{\Theta} L(\Theta | D) = \arg \max_{\Theta} \ell(\Theta | D)$$

NOTE: q doesn't make any impact for first term ℓ_1 (μ_0, μ_1, Σ) doesn't affect second term

$\Rightarrow$ hence, it is removed in $\ell(\Theta | D)$

$$\ell(\Theta | D) = \sum_{i=1}^n \ln(p(x^{(i)} | y^{(i)}, \mu_0, \mu_1, \Sigma)) + \sum_{i=1}^n \ln(p(y^{(i)} | q))$$

$$p(x | y=0) = \frac{1}{(2\pi)^{d/2} (\Sigma)^{1/2}} \underbrace{\exp(\dots)}_{f_0(\mu_0, \Sigma)}$$

$$p(x | y=1) = \frac{1}{(2\pi)^{d/2} (\Sigma)^{1/2}} f_1(\mu_1, \Sigma)$$

$$p(y) = q^y (1-q)^{1-y}$$

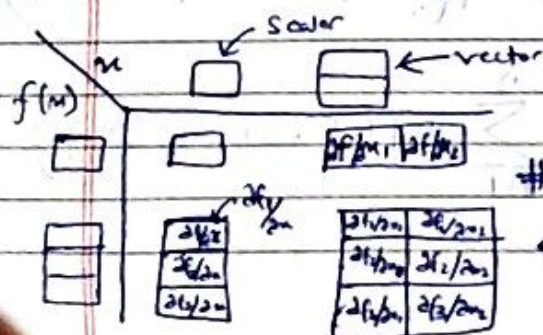
• Now, take gradient

$$\nabla \ell(\Theta | D) = 0 \quad \# \text{ (Calculate this)}$$

$$\Rightarrow \frac{\partial \ell}{\partial \mu_0} = 0, \quad \frac{\partial \ell}{\partial \mu_1} = 0, \quad \frac{\partial \ell}{\partial \Sigma} = 0, \quad \frac{\partial \ell}{\partial q} = 0$$

• How to calculate where vectors involving $\leftarrow$ like matrices

$\rightarrow$ Note: log-likelihood is scalar



How to take derivative

Book

Maths for ML

$\rightarrow$ Foundational

Maths for revision

↓
how
can
dis
h

M: Matrix

(# expressions will be given in exam if at all asked for derivative)

$$i) \frac{\partial M}{\partial \alpha} = \begin{pmatrix} \frac{\partial M_{11}}{\partial \alpha} & \dots & \frac{\partial M_{1d_2}}{\partial \alpha} \\ \vdots & & \vdots \\ \frac{\partial M_{d_1 1}}{\partial \alpha} & \dots & \frac{\partial M_{d_1 d_2}}{\partial \alpha} \end{pmatrix}$$

Scalar

$$ii) \frac{\partial M^{-1}}{\partial \alpha} = -M^{-1} \frac{\partial M}{\partial \alpha} M^{-1}$$

$$iii) \frac{\partial [Mx]}{\partial x} = M$$

$$iv) \frac{\partial (y^T x)}{\partial x} = y$$

(here, y is independent vector of x)

$$v) \frac{\partial (x^T M x)}{\partial x} = (M + M^T)x$$

$$vi) \frac{\partial \log |Z|}{\partial \Sigma} = \Sigma^{-1}$$

$$vii) \frac{\partial (x^T \Sigma x)}{\partial \Sigma} = -\Sigma^{-1} x x^T \Sigma^{-1}$$

Verify the below:

$$1) \hat{q} = \frac{1}{n} \sum_{i=1}^n \mathbb{1}(y^{(i)} = 1)$$

(it tells sum up all samples which belong to class 1)

here features are binary from continuous distribution & have gaussian distribution

$$\hat{y}_0 = \frac{\sum_{i=1}^n x^{(i)} \mathbb{1}(y^{(i)} = 0)}{\sum_{i=1}^n \mathbb{1}(y^{(i)} = 0)}$$

This is sample mean

$A_0: \text{Test } y_i = 0$

$A_1: \text{Test } y_i = 1$

Page No:

Date: / /

$$(iii) \sum = \frac{1}{n} \left[\sum_{i \in A_0} (n^{(i)} - \mu_0) (n^{(i)} - \mu_0)^T + \sum_{i \in A_1} (n^{(i)} - \mu_1) (n^{(i)} - \mu_1)^T \right]$$

Case 1 { # it is also possible that π features are coming from discrete random variables & then estimate parameters
 (of p)
 - here distribution normal or not - (check)
 hence find parameters

Probabilistic Interpretation of Linear Regression
 (the note contains this)

(Binomial distribution)

(if nothing is given, take derivative

of $P(H=61|p)$, & then test

on all possibilities of p) ***

* if p -value < 0.05

$\Rightarrow H_0$ (null hypothesis)

is rejected & the

model is statistically significant

Case 1

example. An unfair coin is flipped 100 times, and 61 heads are observed. The coin either has probability

$\frac{1}{3}$, $\frac{1}{2}$, or $\frac{2}{3}$ of flipping a head each time

it is flipped. Which of the three is the MLE p ?

$\rightarrow$ pmf: $P(n) = \binom{n}{x} p^x (1-p)^{n-x}$, $0 \leq p \leq 1$
 $x = 0, 1, 2, \dots, n$

given, $n=100$, we need to find which $\hat{p}$

$$P(H=61 | p=\frac{1}{3}) = \binom{100}{61} \left(\frac{1}{3}\right)^{61} \left(\frac{2}{3}\right)^{39} = 3.6 \times 10^{-9}$$

$$P(H=61 | p=\frac{1}{2}) = 0.007$$

$$P(H=61 | p=\frac{2}{3}) = 0.040$$

← maximum

hence MLE $p = \frac{2}{3}$

* it should

Compar Model

• How significant is the test?

Hence (1)



* it starts with some assumption. E computes

→ what is normal & binomial distribution?

Page No. _____

Date 23/04/25

$$y_1, (y_1^{(1)} - y_1)^T$$

p-value (that's how likely is this assumption)

Statistical Significance Testing

(i) ~~Strong~~ Null Hypothesis: default assumption

(ii) different methods exist for that (that there is no effect or no difference)

Comparing 2

Models

(a) Naive Solution

Win Ratio: K_A / K

(Probability of A winning)

• How significant is the test?

Classifies R no. of times

$$(K_A \geq K_B)$$

win time of A
win time of B

⇒ why this method is wrong? (bad)

• magnitude of test

• no. of test performed: same ratio for diff no. of test can be achieved

ex. 0.8 ← for n, small or n, large

Hence we go for:

Win Ratio + No. of trials

(i) Sign Test (No. of wins)

→ how many times comparisons done

how many times A has beaten B

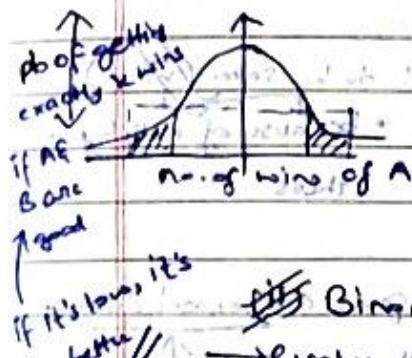
Assumptions



larger the no. of wins

Smaller area under the curve

⇒ more certain that A beats B



Binomial Test

→ p-value tells what is the chance of observing current data when null hypothesis is true

→ lower p-value is good

Compute no. of test done

E. win

plot the binomial distribution

area under

get the tail

→ Final conclusion: p-value (< 0.05) : there is a meaningful difference bet. the performance of A & B

given area under the tail

(more than 530 wins good)

(each value is not the same)

Page No:

Date: / /

Fixed reward based on rank

(ii) Wilcoxon Signed Rank

→ here we consider the magnitude of better as well

→ A B Diff

Performance	A	B	Diff
T ₁	0.7	0.73	-0.03
T ₂	0.8	0.75	0.05
T ₃	0.88	0.81	0.07
T ₄	0.95	0.99	-0.04

Step 1: Arrange difference in the ascending order (ignore sign)

Step 2: Put back the sign
Step 1: Arrange the diff in ascending order (ignoring signs)

Step 3: Rank each entry by rank along with sign
T₁ → T₄ → T₂ → T₃
-1 -2 +3 +4

Step 4: Sum all positive values (W⁺) & negative values (W⁻)
Step 2: Rank them

Step 3: Choose the smaller value as W
Step 4: Choose the smaller value as W

Why? → Both W⁻ & W⁺ should come but to be same (why?)

• Smaller the value
here extreme observation

• farther away from null hypothesis

• because of null hypothesis

Then plot it → gives a normal distribution
(i.e. frequency of that score)

(diff from binomial)

then compute the area under curve

→ use exact magnitude of value

(iii) Permutation Test

(a) Compute diff of performance : then take sum & avg of differences

(b) Do 2^m (at max) change (i.e. A, B value swapped)

for m comparisons
(how?)

Instructions for 1 tailed test

- (c) After each permutation, we get diff avg
 (d) Next find out, which avg are equally or more
 greater or less than $\leftarrow$ extreme than original ^{observation} ~~experiment~~ (ignore the sign)
 This gives, $\rightarrow$ p-value
 both considered for two tailed tests

(Uncertainty == area under the tail)

- (M) Paired T-Test: Assume normal distribution $\rightarrow$ follows t-distribution $\rightarrow$ similar to
 Compare the mean difference between 2 related groups $\rightarrow d_1, d_2, d_3 \dots d_k$ $\leftarrow$ difference observed in performances
 Find mean for $\bar{d}$ (down)
 Find std deviation $\rightarrow$ How many std errors is my observation away from the sample mean

$$t = \frac{\bar{d} - \mu_0}{S_d / \sqrt{K}}$$
 Where $\bar{d}$ = Sampled mean
 S_d = Sample variance
 K = Sample sizes
 Use the normal distribution to find p-value
 if the mean diff is statistically significantly different from zero

- if noisy $\Rightarrow$ use Sign or wilcoxon's
- if clean dataset $\Rightarrow$ use paired t-test

- If p-value very low $\rightarrow$ it tells A will definitely ^{beat} B but not by how much?

$\downarrow$
 told by Cohen's $d = \frac{\bar{d}}{S_d}$

(it tells practical importance)

$\uparrow$ Statistical significance $\neq$ Practical Importance

————— X ————— X —————
 (End of syllabus)

t-distribution: low n or $K \Rightarrow$ heavier tail

higher $K \Rightarrow$ lighter tail

$\infty K \Rightarrow$ normal distribution

- Bias Variance Trade Off
- Performance Measures
- Performance Estimation - on unseen data
- Statistical Significance Testing

Post-Mid

- Clustering
 - K means variants : Quality of seeds imp.
 - Klemung's Theorem
 - Antichustering
 - Evaluation Metric

trick: don't lose : KNN

the initial K - nearest neighbors Intrinsic Extrinsic

PreMid Sem

- Regression
 - ANN - Data req to be in numerical format
 - DBScan
 - Hierarchical Clustering

* (last page contd) (DIY)

→ Decision surface for any linear machine chosen
Such that, the hyperplane is defined by,

$g_i(n) = g_j(n)$ for i categorical with the highest posterior probabilities

$$\text{When } g_i(n) = w_i^T(n) + w_{i0}$$

$$\Rightarrow w^T(n - n_0) = 0$$

$$w = \mu_i - \mu_j$$

(**)

$$w_i^T x + w_{i0} = w_j^T x + w_{j0}$$

$$x_0 = \frac{1}{2} (\mu_i + \mu_j) - \frac{\sigma^2}{\|\mu_i - \mu_j\|^2} \ln \frac{P(w_i)}{P(w_j)} \frac{(\mu_i - \mu_j)}{(\mu_i - \mu_j)}$$

$$(w_i - w_j)^T x = w_{j0} - w_{i0}$$

$$(w_i - w_j)^T x = w_{i0} - w_{j0}$$

$$(w_i - w_j)^T x = (w_i - w_j)^T x_0$$

$$w^T(n - n_0) = 0$$

$$w = \mu_i - \mu_j$$

~~found using $P(w_i/n) > P(w_j/n)$~~
~~& then taking log & rearranging terms~~

Case 2: $\Sigma_i = \Sigma$

$$g_i(x) = -\frac{1}{2} (x - \mu_i)^T \Sigma^{-1} (x - \mu_i) + \ln P(w_i)$$

If $P(w_i) \rightarrow$ Same for all classes, decision rule remains ~~class~~ that chooses the feature vector x to the class with the nearest mean vector of class c

if $P(w_i) \rightarrow$ Liked: the decision is in favor of the prior more likely class

expand: $(x - \mu_i)^T \Sigma^{-1} (x - \mu_i)$

$\Rightarrow x^T \Sigma^{-1} x$ ignore as it is independent of i

$$g_i(x) = w_i^T x + w_{i0}$$

$$w_i = \Sigma^{-1} \mu_i$$

$$w_{i0} = -\frac{1}{2} \mu_i^T \Sigma^{-1} \mu_i + \ln P(w_i)$$

Since the discriminant function is linear, the outcome of the decision boundaries are again, the hyperplane with the equation:

$$w^T (x - x_0) = 0$$

$$w = \Sigma^{-1} (\mu_i - \mu_j)$$

$$x_0 = \frac{1}{2} (\mu_i + \mu_j) - \left\{ \frac{\ln \left[\frac{P(w_i)}{P(w_j)} \right]}{(\mu_i - \mu_j)^T \Sigma^{-1} (\mu_i - \mu_j)} \right\} \times (\mu_i - \mu_j)$$

$\Rightarrow$ The difference this hyperplane has as compared from the one in case 1 is that it is not orthogonal to the line between the means of class. It doesn't intersect halving the point between the means if the priors are equal.

$$(x - \mu_i)^T \Sigma^{-1} (x - \mu_i) = (x - \mu_j)^T \Sigma^{-1} (x - \mu_j)$$

What are we doing? $\rightarrow -P(\cdot) \rightarrow$ Gaussian Models (assumed)

Decision rule: Compute discriminant function
 $\downarrow$
then find the decision boundary
 $\downarrow$
Classify new points

Case 1: $\Sigma_i = \text{arbitrary}$

$\Rightarrow$ only term that is dropped is $\frac{d}{2 \ln 2\pi}$

$\Rightarrow$ resulting discriminant function is no more linear, it is actually quadratic

$$g_i(x) = x^T W_i x + w_{i0} x + w_{i1}$$

$$\text{where, } W_i = -\frac{1}{2} \Sigma^{-1}$$

$$W_i = \Sigma_i^{-1} x_i$$

$$W_{i0} = -\frac{1}{2} x_i^T \Sigma_i^{-1} x_i - \frac{1}{2} \ln |\Sigma_i| + \ln P(w_i)$$

$\Rightarrow$ decision surfaces are hyperquadrics

i.e. general form of hyperplanes, pairs of hyperplanes, hyperspheres, hyperparaboloids, and more freaky shapes

Ex: x_1, x_2, x_3

NOTE: Discriminant function derived using by
** substituting $P(\cdot)$ term & then reduced
to linear or quadratic form

Distributions

a) Binomial: $\binom{n}{x} p^x (1-p)^{n-x}$

b) Bernoulli: $p^{x_i} (1-p)^{1-x_i}$ where $x_i = 1 \text{ or } 0$

take logarithm

$$\text{i.e. } L = \prod p^{x_i} (1-p)^{1-x_i}$$

depending upon what's
suggested

$$\log L = \sum x_i \log p + \sum (1-x_i) \log (1-p)$$

⇒ We are finding the mathematical condition under which a new input should be classified into a particular group, based on probability models for each group.

Page No:

Date: / /

NOTE: # Statistical Significance Test

a) $p(\text{value}) = P(\text{data} | H_0)$

→ $p\text{value} \neq P(H_1)$

→ $p\text{value} \neq P(H_0 | \text{data})$

→ $p\text{value} \neq P(H_0 = \text{false})$

• $P(H_0 | \text{data}) \cdot P(\text{data}) = P(\text{data} | H_0) P(H_0)$

b) pvalue is area under the tails

c) Wilcoxon Signed Rank

Building

Wilcoxon

Test

for small

n

• N comparisons → 2^N possible sign combinations of ranks

• W value close to 0: Reject H_0

• H_0 : if A & B are similar in performance then positive & negative ranks will be alternative

• for high value of N : Approximation with normal distribution

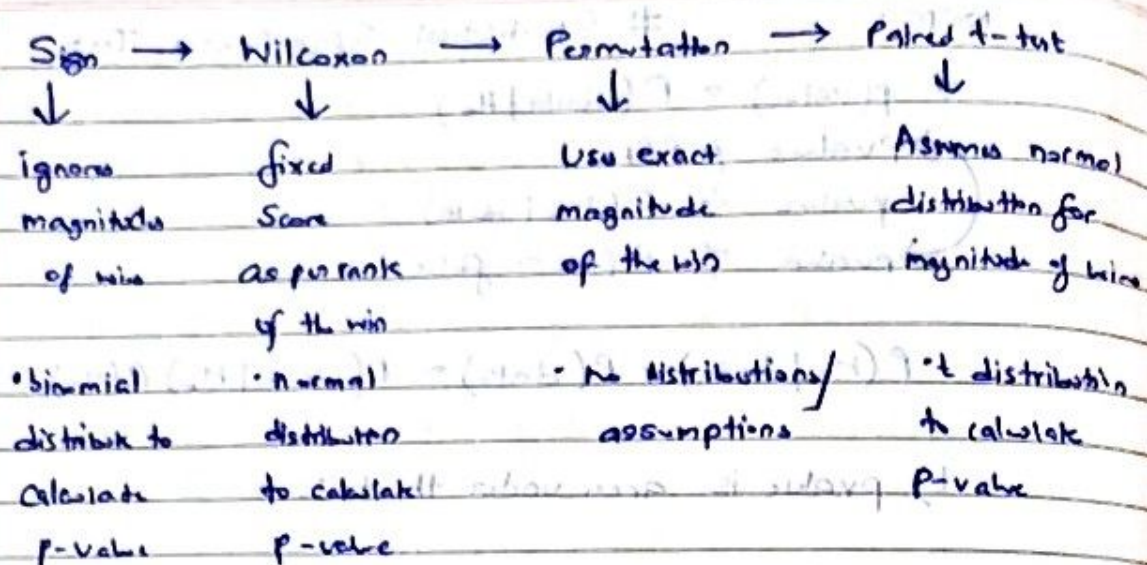
→ Consider only W^+ : Range: 0 to $\frac{N(N+1)}{2}$

$\mu = \frac{N(N+1)}{4}$: Half the ranks are positive

$\sigma = \sqrt{\frac{n(n+1)(n+1)}{24}}$

$Z = \frac{W^+ - \mu}{\sigma}$: How many standard deviations away from mean

$|Z| \geq 1.96 \Rightarrow p\text{value} \leq 0.05$



Increasing sensitivity
noisy observations → Clean observations

* Measures for evaluating binary classifier

- Accuracy: overall my predictions are correct

Pro: • single number summary

• fast computation

Con: • no difference in types of errors

• Class imbalance

• model confidence & calibration

• Threshold training

- Precision: I am confident at identifying positive class

Pro: • fast computation

• Problems where FPs should be minimised

Con: • Ignore FNs

• Can be gained by very rarely predicting the positive class

- Recall: I don't miss many positive instances (Sensitivity)

Pro: • fast computation

• Problems where FNs should be minimised

Can ignore F₁

Can be gamed by labelling everything as positive class

- Specificity: I don't miss any negative instances

Pros: Problems where F₁ should be minimised

- Fast computation

Cons: Ignore F₁

Can be gamed by labelling everything as negative

- Negative Predictive Value: I am confident at identifying negative instances

Pros: Fast computation

Cons: Problems where false negatives should be minimised

Can ignore F₁

Can be gamed by very rarely predicting the negative class

- F1 Score: I can detect most of predictive instances with high confidence in my predictions

Pros: Balance between precision & recall

- Robust to class imbalance

Cons: Sensitive to low precision or low recall

- Ignores T₁

- Matthews Correlation Coefficient

Pros: Considers all entries in the confusion matrix

Cons: Doesn't differentiate between types of errors

Easy interpretability of other measures

- Why Lloyd's algo inefficient?

- distance calculations redundant

(no need to mention exact distances to say whether this point belongs to this cluster)

Given feature Matrix

Properties satisfied by accelerated k-means also

- (a) able to start with any initial centers
- (b) able to use any blackbox distance metric
- (c) given the same initial centers, it should always produce the same final centers as the standard algorithm

The only blackbox property that all distance metric d possess is the Δ Inequality

↑
gives only upper bounds

To get lower bounds,

- If $d(b, c) \geq 2d(n, b) \Rightarrow d(n, c) \geq d(n, b)$
- $d(n, c) \geq \max(0, d(n, b) - d(n, c))$

$L' \leftarrow$ lower bound in prev iteration

$d(n, v) \geq L'$

$L \leftarrow$ for worst iteration:

$$d(n, b) \geq \max(0, d(n, b') - d(b, b'))$$

$$\geq \max(0, L' - d(b, b'))$$

$$\geq L$$

Alkan's $\rightarrow$ (i) At start of each iteration, $U \in L$

are tight for most points & centers

$\Rightarrow$ reducing no. of distance calculations in that iteration

Limitation: (i) lower bounds are updated for all points n & centers $c \leftarrow$ for each iteration

Time: $O(nke)$

Hammarly algo $\rightarrow$ (i) 2 distance bounds per data point for its two closest centers

upper bound on distance to the closest center $\rightarrow$ lower bound on the distance closest to second-closest center